Python code for Artificial Intelligence

Foundations of Computational Agents

David L. Poole and Alan K. Mackworth

Version 0.9.17 of July 7, 2025.

https://aipython.org https://artint.info

Contents

Chapter 1

Python for Artificial Intelligence

AIPython contains runnable code for the book *Artificial Intelligence, foundations of computational agents, 3rd Edition* [Poole and Mackworth, 2023]. It has the following design goals:

- Readability is more important than efficiency, although the asymptotic complexity is not compromised. AIPython is not a replacement for well-designed libraries, or optimized tools. Think of it like a model of an engine made of glass, so you can see the inner workings; don't expect it to power a big truck, but it lets you see how an engine works to power a truck.

- It uses as few libraries as possible. A reader only needs to understand Python. Libraries hide details that we make explicit. The only library used is matplotlib for plotting and drawing.

1.1 Why Python?

We use Python because Python programs can be close to pseudo-code. It is designed for humans to read.

Python is reasonably efficient. Efficiency is usually not a problem for small examples. If your Python code is not efficient enough, a general procedure to improve it is to find out what is taking most of the time, and implement just that part more efficiently in some lower-level language. Many lower-level languages interoperate with Python nicely. This will result in much less programming and more efficient code (because you will have more time to optimize) than writing everything in a lower-level language. Much of the code here is more efficiently implemented in libraries that are more difficult to understand.

1.2 Getting Python

You need Python 3.9 or later (`https://python.org/`) and a compatible version of matplotlib (`https://matplotlib.org/`). This code is *not* compatible with Python 2 (e.g., with Python 2.7).

Download and install the latest Python 3 release from `https://python.org/` or `https://www.anaconda.com/download` (free download includes many libraries). This should also install `pip`. You can install matplotlib using

```
pip install matplotlib
```

in a terminal shell (not in Python). That should "just work". If not, try using `pip3` instead of `pip`.

The command `python` or `python3` should then start the interactive Python shell. You can quit Python with a control-D or with `quit()`.

To upgrade matplotlib to the latest version (which you should do if you install a new version of Python) do:

```
pip install --upgrade matplotlib
```

We recommend using the enhanced interactive python **ipython** (`https://ipython.org/`) [Pérez and Granger, 2007]. To install ipython after you have installed python do:

```
pip install ipython
```

1.3 Running Python

We assume that everything is done with an interactive Python shell. You can either do this with an IDE, such as IDLE that comes with standard Python distributions, or just running ipython or python (or perhaps ipython3 or python3) from a shell.

Here we describe the most simple version that uses no IDE. If you download the zip file, and cd to the "`aipython`" folder where the .py files are, you should be able to do the following, with user input in bold. The first `python` command is in the operating system shell; the `-i` is important to enter interactive mode.

```
python -i searchGeneric.py
Testing problem 1:
7 paths have been expanded and 4 paths remain in the frontier
Path found: A --> C --> B --> D --> G
Passed unit test
>>> searcher2 = AStarSearcher(searchProblem.acyclic_delivery_problem) #A*
>>> searcher2.search()  # find first path
16 paths have been expanded and 5 paths remain in the frontier
o103 --> o109 --> o119 --> o123 --> r123
>>> searcher2.search()  # find next path
```

```
21 paths have been expanded and 6 paths remain in the frontier
o103 --> b3 --> b4 --> o109 --> o119 --> o123 --> r123
>>> searcher2.search()  # find next path
28 paths have been expanded and 5 paths remain in the frontier
o103 --> b3 --> b1 --> b2 --> b4 --> o109 --> o119 --> o123 --> r123
>>> searcher2.search()  # find next path
No (more) solutions. Total of 33 paths expanded.
>>>
```

You can then interact at the last prompt.

There are many textbooks for Python. The best source of information about python is `https://www.python.org/`. The documentation is at `https://docs.python.org/3/`.

The rest of this chapter is about what is special about the code for AI tools. We only use the standard Python library and matplotlib. All of the exercises can be done (and should be done) without using other libraries; the aim is for you to spend your time thinking about how to solve the problem rather than searching for pre-existing solutions.

1.4 Pitfalls

It is important to know when side effects occur. Often AI programs consider what would/might happen given certain conditions. In many such cases, we don't want side effects. When an agent acts in the world, side effects are appropriate.

In Python, you need to be careful to understand side effects. For example, the inexpensive function to add an element to a list, namely `append`, changes the list. In a functional language like Haskell or Lisp, adding a new element to a list, without changing the original list, is a cheap operation. For example if x is a list containing n elements, adding an extra element to the list in Python (using `append`) is fast, but it has the side effect of changing the list x. To construct a new list that contains the elements of x plus a new element, without changing the value of x, entails copying the list, or using a different representation for lists. In the searching code, we will use a different representation for lists for this reason.

1.5 Features of Python

1.5.1 f-strings

Python can use matching `'`, `"`, `'''` or `"""`, the latter two respecting line breaks in the string. We use the convention that when the string denotes a unique symbol, we use single quotes, and when it is designed to be for printing, we use double quotes.

We make extensive use of f-strings https://docs.python.org/3/tutorial/inputoutput.html. In its simplest form

```
f"str1{e1}str2{e2}str3"
```

where e1 and e2 are expressions, is an abbreviation for

```
"str1"+str(e1)+"str2"+str(e2)+"str3"
```

where + is string concatenation, and str is a function that returns a string representation of its argument.

1.5.2 Lists, Tuples, Sets, Dictionaries and Comprehensions

We make extensive uses of lists, tuples, sets and dictionaries (dicts). See https://docs.python.org/3/library/stdtypes.html. Lists use "$[\dots]$", dictionaries use "$\{key : value, \dots\}$", sets use "$\{\dots\}$" (without the :), tuples use "$(\dots)$".

One of the nice features of Python is the use of **comprehensions**: list, tuple, set and dictionary comprehensions.

A list comprehension is of the form

$[fe$ `for` e `in` $iter$ `if` $cond]$

is the list values *fe* for each *e* in *iter* for which *cond* is true. The "`if` *cond*" part is optional, but the "`for`" and "`in`" are not optional. Here *e* is a variable (or a pattern that can be on the left side of =), *iter* is an iterator, which can generate a stream of data, such as a list, a set, a range object (to enumerate integers between ranges) or a file. *cond* is an expression that evaluates to either True or False for each *e*, and *fe* is an expression that will be evaluated for each value of *e* for which *cond* returns `True`. For example:

```
>>> [e*e for e in range(20) if e%2==0]
[0, 4, 16, 36, 64, 100, 144, 196, 256, 324]
```

Comprehensions can also be used for sets and dictionaries. For example, the following creates an index for list a:

```
>>> a = ["a","f","bar","b","a","aaaaa"]
>>> ind = {a[i]:i for i in range(len(a))}
>>> ind
{'a': 4, 'f': 1, 'bar': 2, 'b': 3, 'aaaaa': 5}
>>> ind['b']
3
```

which means that 'b' is the element with index 3 in the list.

The assignment of ind could have also be written as:

```
>>> ind = {val:i for (i,val) in enumerate(a)}
```

where enumerate is a built-in function that, given a dictionary, returns an generator of $(index, value)$ pairs.

1.5.3 Generators

Python has generators which can be used for a form of lazy evaluation – only computing values when needed.

A comprehension in round parentheses gives a generator that can generate the elements as needed. The result can go in a list or used in another comprehension, or can be called directly using `next`. The procedure `next` takes an iterator and returns the next element (advancing the iterator); it raises a StopIteration exception if there is no next element. The following shows a simple example, where user input is prepended with >>>

```
>>> a = (e*e for e in range(20) if e%2==0)
>>> next(a)
0
>>> next(a)
4
>>> next(a)
16
>>> list(a)
[36, 64, 100, 144, 196, 256, 324]
>>> next(a)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

Notice how `list(a)` continued on the enumeration, and got to the end of it.

To make a procedure into a generator, the `yield` command returns a value that is obtained with `next`. It is typically used to enumerate the values for a `for` loop or in generators. (The `yield` command can also be used for coroutines, but AIPython only uses it for generators.)

A version of the built-in `range`, with 2 or 3 arguments (and positive steps) can be implemented as:[1]

pythonDemo.py — Some tricky examples

```
11 def myrange(start, stop, step=1):
12     """enumerates the values from start in steps of size step that are
13     less than stop.
14     """
15     assert step>0, f"only positive steps implemented in myrange: {step}"
16     i = start
17     while i<stop:
18         yield i
19         i += step
20
21 print("list(myrange(2,30,3)):",list(myrange(2,30,3)))
```

[1] Numbered lines are Python code available in the code-directory, `aipython`. The name of the file is given in the gray text above the listing. The numbers correspond to the line numbers in that file.

The built-in `range` is unconventional in how it handles a single argument, as the single argument acts as the second argument of the function. The built-in `range` also allows for indexing (e.g., `range(2,30,3)[2]` returns 8), but the above implementation does not. However `myrange` also works for floats, whereas the built-in `range` does not.

Exercise 1.1 Implement a version of `myrange` that acts like the built-in version when there is a single argument. (Hint: make the second argument have a default value that can be recognized in the function.) There is no need to make it work with indexing.

Yield can be used to generate the same sequence of values as in the example above.

pythonDemo.py — (continued)

```
23 def ga(n):
24     """generates square of even nonnegative integers less than n"""
25     for e in range(n):
26         if e%2==0:
27             yield e*e
28 a = ga(20)
```

The sequence of next(a), and list(a) gives exactly the same results as the comprehension at the start of this section.

It is straightforward to write a version of the built-in `enumerate` called `myenumerate`:

pythonDemo.py — (continued)

```
30 def myenumerate(iter, start=0):
31     i = start
32     for e in iter:
33         yield i,e
34         i += 1
```

1.5.4 Functions as first-class objects

Python can create lists and other data structures that contain functions. There is an issue that tricks many newcomers to Python. For a local variable in a function, the function uses the last value of the variable when the function is *called*, not the value of the variable when the function was defined (this is called "late binding"). This means if you want to use the value a variable has when the function is created, you need to save the current value of that variable. Whereas Python uses "late binding" by default, the alternative that newcomers often expect is "early binding", where a function uses the value a variable had when the function was defined. The following examples show how early binding can be implemented.

Consider the following programs designed to create a list of 5 functions, where the ith function in the list is meant to add i to its argument:

pythonDemo.py — (continued)

```
36 fun_list1 = []
37 for i in range(5):
38     def fun1(e):
39         return e+i
40     fun_list1.append(fun1)
41
42 fun_list2 = []
43 for i in range(5):
44     def fun2(e,iv=i):
45         return e+iv
46     fun_list2.append(fun2)
47
48 fun_list3 = [lambda e: e+i for i in range(5)]
49
50 fun_list4 = [lambda e,iv=i: e+iv for i in range(5)]
51
52 i=56
```

Try to predict, and then test to see the output, of the output of the following calls, remembering that the function uses the latest value of any variable that is not bound in the function call:

pythonDemo.py — (continued)

```
54 # in Shell do
55 ## ipython -i pythonDemo.py
56 # Try these (copy text after the comment symbol and paste in the Python
       prompt):
57 # print([f(10) for f in fun_list1])
58 # print([f(10) for f in fun_list2])
59 # print([f(10) for f in fun_list3])
60 # print([f(10) for f in fun_list4])
```

In the first for-loop, the function `fun1` uses `i`, whose value is the last value it was assigned. In the second loop, the function `fun2` uses `iv`. There is a separate `iv` variable for each function, and its value is the value of `i` when the function was defined. Thus `fun1` uses late binding, and `fun2` uses early binding. `fun_list3` and `fun_list4` are equivalent to the first two (except `fun_list4` uses a different `i` variable).

One of the advantages of using the embedded definitions (as in `fun1` and `fun2` above) over the lambda is that is it possible to add a `__doc__` string, which is the standard for documenting functions in Python, to the embedded definitions.

1.6 Useful Libraries

1.6.1 Timing Code

In order to compare algorithms, you may want to compute how long a program takes to run; this is called the **run time** of the program. The most straightforward way to compute the run time of `foo.bar(aaa)` is to use `time.perf_counter()`, as in:

```
import time
start_time = time.perf_counter()
foo.bar(aaa)
end_time = time.perf_counter()
print("Time:", end_time - start_time, "seconds")
```

Note that `time.perf_counter()` measures clock time; so this should be done without user interaction between the calls. On the interactive python shell, you should do:

```
start_time = time.perf_counter(); foo.bar(aaa); end_time = time.perf_counter()
```

If this time is very small (say less than 0.2 second), it is probably very inaccurate; run your code multiple times to get a more accurate count. For this you can use `timeit` (`https://docs.python.org/3/library/timeit.html`). To use timeit to time the call to `foo.bar(aaa)` use:

```
import timeit
time = timeit.timeit("foo.bar(aaa)",
                     setup="from __main__ import foo,aaa", number=100)
```

The setup is needed so that Python can find the meaning of the names in the string that is called. This returns the number of seconds to execute `foo.bar(aaa)` 100 times. The `number` should be set so that the run time is at least 0.2 seconds.

You should not trust a single measurement as that can be confounded by interference from other processes. `timeit.repeat` can be used for running `timeit` a few (say 3) times. When reporting the time of any computation, you should be explicit and explain what you are reporting. Usually the minimum time is the one to report (as it is the run with less interference).

1.6.2 Plotting: Matplotlib

The standard plotting for Python is matplotlib (`https://matplotlib.org/`). We will use the most basic plotting using the pyplot interface.

Here is a simple example that uses most of AIPython uses. The output is shown in Figure 1.1.

pythonDemo.py — (continued)

```
62 import matplotlib.pyplot as plt
63
```

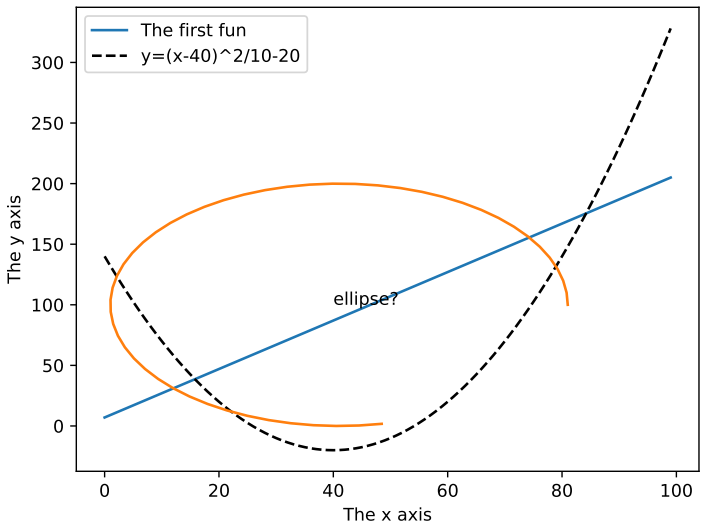


Figure 1.1: Result of pythonDemo code

```
64 def myplot(minv,maxv,step,fun1,fun2):
65     global fig, ax # allow them to be used outside myplot()
66     plt.ion() # make it interactive
67     fig, ax = plt.subplots()
68     ax.set_xlabel("The x axis")
69     ax.set_ylabel("The y axis")
70     ax.set_xscale('linear') # Makes a 'log' or 'linear' scale
71     xvalues = range(minv,maxv,step)
72     ax.plot(xvalues,[fun1(x) for x in xvalues],
73               label="The first fun")
74     ax.plot(xvalues,[fun2(x) for x in xvalues], linestyle='--',color='k',
75               label=fun2.__doc__) # use the doc string of the function
76     ax.legend(loc="upper right") # display the legend
77
78 def slin(x):
79     """y=2x+7"""
80     return 2*x+7
81 def sqfun(x):
82     """y=(x-40)^2/10-20"""
83     return (x-40)**2/10-20
84
85 # Try the following from shell:
86 # python -i pythonDemo.py
87 # myplot(0,100,1,slin,sqfun)
88 # ax.legend(loc="best")
89 # import math
90 # ax.plot([41+40*math.cos(th/10) for th in range(50)],
```

```
#         [100+100*math.sin(th/10) for th in range(50)])
# ax.text(40,100,"ellipse?")
# ax.set_xscale('log')
```

At the end of the code are some commented-out commands you should try in interactive mode. Cut from the file and paste into Python (and remember to remove the comments symbol and leading space).

1.7 Utilities

1.7.1 Display

To keep things simple, using only standard Python, AIPython code is written using a text-oriented tracing.

The method `self.display` is used to trace the program. Any call

`self.display`(*level*, *to_print* ...)

where the *level* is less than or equal to the value for `max_display_level` will be printed. The *to_print* ... can be anything that is accepted by the built-in `print` (including any keyword arguments).

The definition of `display` is:

display.py — A simple way to trace the intermediate steps of algorithms.

```
class Displayable(object):
    """Class that uses 'display'.
    The amount of detail is controlled by max_display_level
    """
    max_display_level = 1 # can be overridden in subclasses or instances

    def display(self,level,*args,**nargs):
        """print the arguments if level is less than or equal to the
        current max_display_level.
        level is an integer.
        the other arguments are whatever arguments print can take.
        """
        if level <= self.max_display_level:
            print(*args, **nargs) ##if error you are using Python2 not
                Python3
```

In this code, `args` gets a tuple of the positional arguments, and `nargs` gets a dictionary of the keyword arguments. This will not work in Python 2, and will give an error.

Any class that wants to use `display` can be made a subclass of `Displayable`.

To change the maximum display level to 3 for a class do:

Classname.`max_display_level` = 3

which will make calls to `display` in that class print when the value of `level` is less-than-or-equal to 3. The default display level is 1. It can also be changed for individual objects (the object value overrides the class value).

The value of `max_display_level` by convention is:

0 display nothing

1 display solutions (nothing that happens repeatedly)

2 also display the values as they change (little detail through a loop)

3 also display more details

4 and above even more detail

To implement a graphical user interface (GUI), the definition of `display` can be overridden. See, for example, `SearcherGUI` in Section 3.2.2 and `ConsistencyGUI` in Section 4.4.2. These GUIs use the AIPython code unchanged.

1.7.2 Argmax

Python has a built-in `max` function that takes a generator (or a list or set) and returns the maximum value. The `argmaxall` method takes a generator of (*element*, *value*) pairs, as for example is generated by the built-in `enumerate`(*list*) for lists or *dict*`.items()` for dictionaries. It returns a list of all elements with maximum value; `argmaxe` returns one of these values at random. The `argmax` method takes a list and returns the index of a random element that has the maximum value. `argmaxd` takes a dictionary and returns a key with maximum value.

utilities.py — AIPython useful utilities

```
11 import random
12 import math
13
14 def argmaxall(gen):
15     """gen is a generator of (element,value) pairs, where value is a real.
16     argmaxall returns a list of all of the elements with maximal value.
17     """
18     maxv = -math.inf     # negative infinity
19     maxvals = []    # list of maximal elements
20     for (e,v) in gen:
21         if v > maxv:
22             maxvals, maxv = [e], v
23         elif v == maxv:
24             maxvals.append(e)
25     return maxvals
26
27 def argmaxe(gen):
28     """gen is a generator of (element,value) pairs, where value is a real.
29     argmaxe returns an element with maximal value.
```

```
30      If there are multiple elements with the max value, one is returned at
            random.
31      """
32      return random.choice(argmaxall(gen))
33
34  def argmax(lst):
35      """returns maximum index in a list"""
36      return argmaxe(enumerate(lst))
37  # Try:
38  # argmax([1,6,3,77,3,55,23])
39
40  def argmaxd(dct):
41     """returns the arg max of a dictionary dct"""
42     return argmaxe(dct.items())
43  # Try:
44  # arxmaxd({2:5,5:9,7:7})
```

Exercise 1.2 Change `argmaxe` to have an optional argument that specifies whether you want the "first", "last" or a "random" index of the maximum value returned. If you want the first or the last, you don't need to keep a list of the maximum elements. Enable the other methods to have this optional argument, if appropriate.

1.7.3 Probability

For many of the simulations, we want to make a variable True with some probability. `flip(p)` returns True with probability `p`, and otherwise returns False.

utilities.py — (continued)

```
45  def flip(prob):
46      """return true with probability prob"""
47      return random.random() < prob
```

The `select_from_dist` method takes in a *item* : *probability* dictionary, and returns one of the items in proportion to its probability. The probabilities should sum to 1 or more. If they sum to more than one, the excess is ignored.

utilities.py — (continued)

```
49  def select_from_dist(item_prob_dist):
50      """ returns a value from a distribution.
51      item_prob_dist is an item:probability dictionary, where the
52          probabilities sum to 1.
53      returns an item chosen in proportion to its probability
54      """
55      ranreal = random.random()
56      for (it,prob) in item_prob_dist.items():
57          if ranreal < prob:
58              return it
59          else:
60              ranreal -= prob
61      raise RuntimeError(f"{item_prob_dist} is not a probability
            distribution")
```

1.8 Testing Code

It is important to test code early and test it often. We include a simple form of **unit test**. In your code, you should do more substantial testing than done here. Make sure you should also test boundary cases.

The following code tests `argmax`, but only if `utilities` is loaded in the top-level. If it is loaded in a module the test code is not run. The value of the current module is in `__name__` and if the module is run at the top-level, its value is `"__main__"`. See `https://docs.python.org/3/library/__main__.html`.

utilities.py — (continued)

```
63 def test():
64     """Test part of utilities"""
65     assert argmax([1,6,55,3,55,23]) in [2,4]
66     print("Passed unit test in utilities")
67     print("run test_aipython() to test (almost) everything")
68
69 if __name__ == "__main__":
70     test()
```

The following imports all of the python code and does a simple check of all of AIPython that has automatic checks. If you develop new algorithms or tests, add them here!

utilities.py — (continued)

```
72 def test_aipython():
73     import pythonDemo, display
74     # Agents: currently no tests
75     import agents, agentBuying, agentEnv, agentMiddle, agentTop,
           agentFollowTarget
76     # Search:
77     print("***** testing Search *****")
78     import searchGeneric, searchBranchAndBound, searchExample, searchTest
79     searchGeneric.test(searchGeneric.AStarSearcher)
80     searchBranchAndBound.test(searchBranchAndBound.DF_branch_and_bound)
81     searchTest.run(searchExample.problem1,"Problem 1")
82     import searchGUI, searchMPP, searchGrid
83     # CSP
84     print("\n***** testing CSP *****")
85     import cspExamples, cspDFS, cspSearch, cspConsistency, cspSLS
86     cspExamples.test_csp(cspDFS.dfs_solve1)
87     cspExamples.test_csp(cspSearch.solver_from_searcher)
88     cspExamples.test_csp(cspConsistency.ac_solver)
89     cspExamples.test_csp(cspConsistency.ac_search_solver)
90     cspExamples.test_csp(cspSLS.sls_solver)
91     cspExamples.test_csp(cspSLS.any_conflict_solver)
92     import cspConsistencyGUI, cspSoft
93     # Propositions
94     print("\n***** testing Propositional Logic *****")
```

```
    import logicBottomUp, logicTopDown, logicExplain, logicAssumables,
        logicNegation
    logicBottomUp.test()
    logicTopDown.test()
    logicExplain.test()
    logicNegation.test()
    # Planning
    print("\n***** testing Planning *****")
    import stripsHeuristic
    stripsHeuristic.test_forward_heuristic()
    stripsHeuristic.test_regression_heuristic()
    import stripsCSPPlanner, stripsPOP
    # Learning
    print("\n***** Learning with no inputs *****")
    import learnProblem, learnNoInputs, learnDT, learnLinear
    learnNoInputs.test_no_inputs(training_sizes=[4])
    data = learnProblem.Data_from_file('data/carbool.csv', one_hot=True,
        target_index=-1, seed=123)
    print("\n***** Decision Trees *****")
    learnDT. DT_learner(data).evaluate()
    print("\n***** Linear Learning *****")
    learnLinear.Linear_learner(data).evaluate()
    import learnCrossValidation, learnBoosting
    # Deep Learning
    import learnNN
    print("\n***** testing Neural Network Learning *****")
    learnNN.NN_from_arch(data, arch=[3]).evaluate()
    # Uncertainty
    print("\n***** testing Uncertainty *****")
    import probGraphicalModels, probRC, probVE, probStochSim
    probGraphicalModels.InferenceMethod.testIM(probRC.ProbSearch)
    probGraphicalModels.InferenceMethod.testIM(probRC.ProbRC)
    probGraphicalModels.InferenceMethod.testIM(probVE.VE)
    probGraphicalModels.InferenceMethod.testIM(probStochSim.RejectionSampling,
        threshold=0.1)
    probGraphicalModels.InferenceMethod.testIM(probStochSim.LikelihoodWeighting,
        threshold=0.1)
    probGraphicalModels.InferenceMethod.testIM(probStochSim.ParticleFiltering,
        threshold=0.1)
    probGraphicalModels.InferenceMethod.testIM(probStochSim.GibbsSampling,
        threshold=0.1)
    import probHMM, probLocalization, probDBN
    # Learning under uncertaint
    print("\n***** Learning under Uncertainty *****")
    import learnBayesian, learnKMeans, learnEM
    learnKMeans.testKM()
    learnEM.testEM()
    # Causality: currently no tests
    import probDo, probCounterfactual
    # Planning under uncertainty
```

```
    print("\n***** Planning under Uncertainty *****")
    import decnNetworks
    decnNetworks.test(decnNetworks.fire_dn)
    import mdpExamples
    mdpExamples.test_MDP(mdpExamples.partyMDP)
    import mdpGUI
    # Reinforcement Learning:
    print("\n***** testing Reinforcement Learning *****")
    import rlQLearner
    rlQLearner.test_RL(rlQLearner.Q_learner, alpha_fun=lambda k:10/(9+k))
    import rlQExperienceReplay
    rlQLearner.test_RL(rlQExperienceReplay.Q_ER_learner, alpha_fun=lambda
        k:10/(9+k))
    import rlStochasticPolicy
    rlQLearner.test_RL(rlStochasticPolicy.StochasticPIAgent,
        alpha_fun=lambda k:10/(9+k))
    import rlModelLearner
    rlQLearner.test_RL(rlModelLearner.Model_based_reinforcement_learner)
    import rlFeatures
    rlQLearner.test_RL(rlFeatures.SARSA_LFA_learner,
        es_kwargs={'epsilon':1}, eps=4)
    import rlQExperienceReplay, rlModelLearner, rlFeatures, rlGUI
    # Multiagent systems: currently no tests
    import rlStochasticPolicy, rlGameFeature
    # Individuals and Relations
    print("\n***** testing Datalog and Logic Programming *****")
    import relnExamples
    relnExamples.test_ask_all()
    # Knowledge Graphs and Ontologies
    print("\n***** testing Knowledge Graphs and Ontologies *****")
    import knowledgeGraph, knowledgeReasoning
    knowledgeGraph.test_kg()
    # Relational Learning: currently no tests
    import relnCollFilt, relnProbModels
    print("\n***** End of Testing*****")
```

Chapter 2

Agent Architectures and Hierarchical Control

This implements the controllers described in Chapter 2 of Poole and Mackworth [2023]. It defines an architecture that is also used by reinforcement learning (Chapter 13) and multiagent learning (Section 14.2).

AIPython only provides sequential implementations of the control. More sophisticated version may have them run concurrently. Higher-levels call lower-levels. The higher-levels calling the lower-level works in simulated environments where the lower-level are written to make sure they return (and don't go on forever), and the higher level doesn't take too long (as the lower-levels will wait until called again). More realistic architecture have the layers running concurrently so the lower layer can keep reacting while the higher layers are carrying out more complex computation.

2.1 Representing Agents and Environments

Both agents and the environment are treated as objects in the sense of object-oriented programming, with an internal state they maintain, and can evaluate methods. In this chapter, only a single agent is allowed; Section 14.2 allows for multiple agents.

An **environment** takes in actions of the agents, updates its internal state and returns the next percept, using the method `do`.

An **agent** implements the method `select_action` that takes a percept and returns the next action, updating its internal state as appropriate.

The methods `do` and `select_action` are chained together to build a simulator. Initially the simulator needs either an action or a percept. There are two variants used:

- An agent implements the `initial_action(percept)` method which is used initially. This is the method used in the reinforcement learning chapter (page 319).
- The environment implements the `initial_percept()` method which gives the initial percept for the agent. This is the method is used in this chapter.

The state of the agent and the state of the environment are represented using standard Python variables, which are updated as the state changes. The percept and the actions are represented as variable-value dictionaries.

`Agent` and `Environment` are subclasses of `Displayable` so that they can use the `display` method described in Section 1.7.1. `raise NotImplementedError()` is a way to specify an abstract method that needs to be overridden in any implemented agent or environment.

agents.py — Agent and Controllers

```
from display import Displayable

class Agent(Displayable):

    def initial_action(self, percept):
        """return the initial action."""
        return self.select_action(percept) # same as select_action

    def select_action(self, percept):
        """return the next action (and update internal state) given percept
        percept is variable:value dictionary
        """
        raise NotImplementedError("go") # abstract method
```

The environment implements a `do(action)` method where `action` is a variable-value dictionary. This returns a percept, which is also a variable-value dictionary. The use of dictionaries allows for structured actions and percepts.

Note that

agents.py — (continued)

```
class Environment(Displayable):
    def initial_percept(self):
        """returns the initial percept for the agent"""
        raise NotImplementedError("initial_percept") # abstract method

    def do(self, action):
        """does the action in the environment
        returns the next percept """
        raise NotImplementedError("Environment.do") # abstract method
```

The simulator is initialized with `initial_percept` and then the agent and the environment take turns in updating their states and returning the action and the percept. This simulator runs for n steps. A slightly more sophisticated simulator could run until some stopping condition.

agents.py — (continued)

```
class Simulate(Displayable):
    """simulate the interaction between the agent and the environment
    for n time steps.
    """
    def __init__(self, agent, environment):
        self.agent = agent
        self.env = environment
        self.percept = self.env.initial_percept()
        self.percept_history = [self.percept]
        self.action_history = []

    def go(self, n):
        for i in range(n):
            action = self.agent.select_action(self.percept)
            self.display(2,f"i={i} action={action}")
            self.percept = self.env.do(action)
            self.display(2,f"    percept={self.percept}")
```

2.2 Paper buying agent and environment

To run the demo, in folder "aipython", load "agents.py", using e.g., `ipython -i agentBuying.py`, and copy and paste the commented-out commands at the bottom of that file.

This is an implementation of Example 2.1 of Poole and Mackworth [2023]. You might get different plots to Figures 2.2 and 2.3 as there is randomness in the environment.

2.2.1 The Environment

The environment state is given in terms of the `time` and the amount of paper in `stock`. It also remembers the in-stock history and the price history. The percept consists of the price and the amount of paper in stock. The action of the agent is the number to buy.

Here we assume that the price changes are obtained from the `price_delta` list which gives the change in price for each time. When the time is longer than the list, it repeats the list. Note that the sum of the changes is greater than zero, so that prices tend to increase. There is also randomness (noise) added to the prices. The agent cannot access the price model; it just observes the prices and the amount in stock.

agentBuying.py — Paper-buying agent

```
import random
from agents import Agent, Environment, Simulate
from utilities import select_from_dist
```

```
14
15 class TP_env(Environment):
16     price_delta = [0, 0, 0, 21, 0, 20, 0, -64, 0, 0, 23, 0, 0, 0, -35,
17         0, 76, 0, -41, 0, 0, 0, 21, 0, 5, 0, 5, 0, 0, 0, 5, 0, -15, 0, 5,
18       0, 5, 0, -115, 0, 115, 0, 5, 0, -15, 0, 5, 0, 5, 0, 0, 0, 5, 0,
19       -59, 0, 44, 0, 5, 0, 5, 0, 0, 0, 5, 0, -65, 50, 0, 5, 0, 5, 0, 0,
20       0, 5, 0]
21     sd = 5 # noise standard deviation
22
23     def __init__(self):
24         """paper buying agent"""
25         self.time=0
26         self.stock=20
27         self.stock_history = [] # memory of the stock history
28         self.price_history = [] # memory of the price history
29
30     def initial_percept(self):
31         """return initial percept"""
32         self.stock_history.append(self.stock)
33         self.price = round(234+self.sd*random.gauss(0,1))
34         self.price_history.append(self.price)
35         return {'price': self.price,
36                 'instock': self.stock}
37
38     def do(self, action):
39         """does action (buy) and returns percept consisting of price and
               instock"""
40         used = select_from_dist({6:0.1, 5:0.1, 4:0.1, 3:0.3, 2:0.2, 1:0.2})
41         # used = select_from_dist({7:0.1, 6:0.2, 5:0.2, 4:0.3, 3:0.1,
               2:0.1}) # uses more paper
42         bought = action['buy']
43         self.stock = self.stock+bought-used
44         self.stock_history.append(self.stock)
45         self.time += 1
46         self.price = round(self.price
47                            + self.price_delta[self.time%len(self.price_delta)] #
                               repeating pattern
48                            + self.sd*random.gauss(0,1)) # plus randomness
49         self.price_history.append(self.price)
50         return {'price': self.price,
51                 'instock': self.stock}
```

2.2.2 The Agent

The agent does not have access to the price model but can only observe the current price and the amount in stock. It has to decide how much to buy.

The belief state of the agent is an estimate of the average price of the paper, and the total amount of money the agent has spent.

agentBuying.py — (continued)

```
class TP_agent(Agent):
    def __init__(self):
        self.spent = 0
        percept = env.initial_percept()
        self.ave = self.last_price = percept['price']
        self.instock = percept['instock']
        self.buy_history = []

    def select_action(self, percept):
        """return next action to carry out
        """
        self.last_price = percept['price']
        self.ave = self.ave+(self.last_price-self.ave)*0.05
        self.instock = percept['instock']
        if self.last_price < 0.9*self.ave and self.instock < 60:
            tobuy = 48
        elif self.instock < 12:
            tobuy = 12
        else:
            tobuy = 0
        self.spent += tobuy*self.last_price
        self.buy_history.append(tobuy)
        return {'buy': tobuy}
```

Set up an environment and an agent. Uncomment the last lines to run the agent for 90 steps, and determine the average amount spent.

agentBuying.py — (continued)

```
env = TP_env()
ag = TP_agent()
sim = Simulate(ag,env)
#sim.go(90)
#ag.spent/env.time ## average spent per time period
```

2.2.3 Plotting

The following plots the price and number in stock history:

agentBuying.py — (continued)

```
import matplotlib.pyplot as plt

class Plot_history(object):
    """Set up the plot for history of price and number in stock"""
    def __init__(self, ag, env):
        self.ag = ag
        self.env = env
        plt.ion()
        fig, self.ax = plt.subplots()
        self.ax.set_xlabel("Time")
        self.ax.set_ylabel("Value")
```

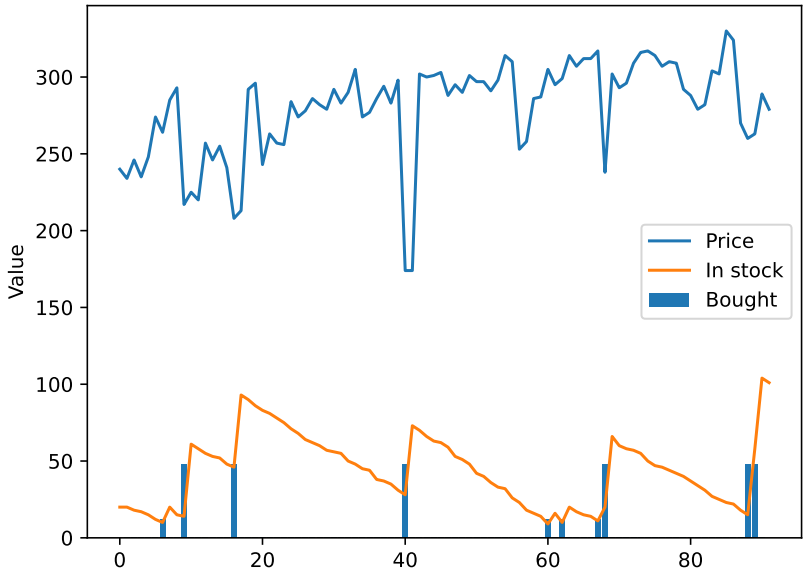


Figure 2.1: Percept and command traces for the paper-buying agent

```
94
95      def plot_env_hist(self):
96          """plot history of price and instock"""
97          num = len(env.stock_history)
98          self.ax.plot(range(num),env.price_history,label="Price")
99          self.ax.plot(range(num),env.stock_history,label="In stock")
100         self.ax.legend()
101
102     def plot_agent_hist(self):
103         """plot history of buying"""
104         num = len(ag.buy_history)
105         self.ax.bar(range(1,num+1), ag.buy_history, label="Bought")
106         self.ax.legend()
107
108 # sim.go(100); print(f"agent spent ${ag.spent/100}")
109 # pl = Plot_history(ag,env); pl.plot_env_hist(); pl.plot_agent_hist()
```

Figure 2.1 shows the result of the plotting in the previous code.

Exercise 2.1 Design a better controller for a paper-buying agent.

- Justify a performance measure that is a fair comparison. Note that minimizing the total amount of money spent may be unfair to agents who have built up a stockpile, and favors agents that end up with no paper.
- Give a controller that can work for many different price histories. An agent can use other local state variables, but does not have access to the environment model.

- Is it worthwhile trying to infer the amount of paper that the home uses? (Try your controller with the different paper consumption commented out in `TP_env.do`.)

2.3 Hierarchical Controller

To run the hierarchical controller, in folder "`aipython`", load "`agentTop.py`", using e.g., `ipython -i agentTop.py`, and copy and paste the commands near the bottom of that file.

In this implementation, each layer, including the top layer, implements the environment class, because each layer is seen as an environment from the layer above.

The robot controller is decomposed as follows. The world defines the walls. The body describes the robot's position, and its physical abilities such as whether its whisker sensor of on. The body can be told to steer left or right or to go straight. The middle layer can be told to go to *x*-*y* positions, avoiding walls. The top layer knows about named locations, such as the storage room and location `o103`, and their *x*-*y* positions. It can be told a sequence of locations, and tells the middle layer to go to the positions of the locations in turn.

2.3.1 Body

`Rob_body` defines everything about the agent body, its position and orientation and whether its whisker sensor is on. It implements the Environment class as it is treated as an environment by the higher layers. It can be told to turn left or right or to go straight.

agentEnv.py — Agent environment

```
11 import math
12 from agents import Environment
13 import matplotlib.pyplot as plt
14 import time
15
16 class Rob_body(Environment):
17     def __init__(self, world, init_pos=(0,0), init_dir=90):
18         """ world is the current world
19         init_pos is a pair of (x-position, y-position)
20         init_dir is a direction in degrees; 0 is to right, 90 is
               straight-up, etc
21         """
22         self.world = world
23         self.rob_pos = init_pos
24         self.rob_dir = init_dir
25         self.turning_angle = 18 # degrees that a left makes
26         self.whisker_length = 6 # length of the whisker
27         self.whisker_angle = 30 # angle of whisker relative to robot
```

```
28          self.crashed = False
29
30      def percept(self):
31          return {'rob_pos':self.rob_pos,
32                  'rob_dir':self.rob_dir, 'whisker':self.whisker(),
                        'crashed':self.crashed}
33      initial_percept = percept # use percept function for initial percept too
34
35      def do(self, action):
36          """ action is {'steer':direction}
37          direction is 'left', 'right' or 'straight'.
38          Returns current percept.
39          """
40          if self.crashed:
41              return self.percept()
42          direction = action['steer']
43          compass_deriv =
                {'left':1,'straight':0,'right':-1}[direction]*self.turning_angle
44          self.rob_dir = (self.rob_dir + compass_deriv +360)%360 # make in
                range [0,360)
45          x,y = self.rob_pos
46          rob_pos_new = (x + math.cos(self.rob_dir*math.pi/180),
47                         y + math.sin(self.rob_dir*math.pi/180))
48          path = (self.rob_pos,rob_pos_new)
49          if any(line_segments_intersect(path,wall) for wall in
                self.world.walls):
50              self.crashed = True
51          self.rob_pos = rob_pos_new
52          self.world.do({'rob_pos':self.rob_pos,
53                         'crashed':self.crashed, 'whisker':self.whisker()})
54          return self.percept()
```

The Boolean `whisker` method returns True when the the robots whisker sensor intersects with a wall.

agentEnv.py — (continued)

```
56      def whisker(self):
57          """returns true whenever the whisker sensor intersects with a wall
58          """
59          whisk_ang_world = (self.rob_dir-self.whisker_angle)*math.pi/180
60              # angle in radians in world coordinates
61          (x,y) = self.rob_pos
62          wend = (x + self.whisker_length * math.cos(whisk_ang_world),
63                  y + self.whisker_length * math.sin(whisk_ang_world))
64          whisker_line = (self.rob_pos, wend)
65          hit = any(line_segments_intersect(whisker_line,wall)
66                    for wall in self.world.walls)
67          return hit
68
69  def line_segments_intersect(linea, lineb):
70      """returns true if the line segments, linea and lineb intersect.
```

```
    A line segment is represented as a pair of points.
    A point is represented as a (x,y) pair.
    """
    ((x0a,y0a),(x1a,y1a)) = linea
    ((x0b,y0b),(x1b,y1b)) = lineb
    da, db = x1a-x0a, x1b-x0b
    ea, eb = y1a-y0a, y1b-y0b
    denom = db*ea-eb*da
    if denom==0:  # line segments are parallel
        return False
    cb = (da*(y0b-y0a)-ea*(x0b-x0a))/denom # intersect along line b
    if cb<0 or cb>1:
        return False  # intersect is outside line segment b
    ca = (db*(y0b-y0a)-eb*(x0b-x0a))/denom # intersect along line a
    return 0<=ca<=1 # intersect is inside both line segments

# Test cases:
# assert line_segments_intersect(((0,0),(1,1)),((1,0),(0,1)))
# assert not line_segments_intersect(((0,0),(1,1)),((1,0),(0.6,0.4)))
# assert line_segments_intersect(((0,0),(1,1)),((1,0),(0.4,0.6)))
```

2.3.2 Middle Layer

The middle layer acts like both a controller (for the body layer) and an environment for the upper layer. It has to tell the body how to steer. Thus it calls *env.do*(·), where env is the body. It implements do(\cdot) for the top layer, where the action specifies an *x*-*y* position to go to and a timeout.

agentMiddle.py — Middle Layer

```
from agents import Environment
import math

class Rob_middle_layer(Environment):
    def __init__(self, lower):
        """The lower-level for the middle layer is the body.
        """
        self.lower = lower
        self.percept = lower.initial_percept()
        self.straight_angle = 11 # angle that is close enough to straight
            ahead
        self.close_threshold = 1 # distance that is close enough to arrived
        self.close_threshold_squared = self.close_threshold**2 # just
            compute it once

    def initial_percept(self):
        return {}

    def do(self, action):
        """action is {'go_to':target_pos,'timeout':timeout}
```

```
29          target_pos is (x,y) pair
30          timeout is the number of steps to try
31          returns {'arrived':True} when arrived is true
32              or {'arrived':False} if it reached the timeout
33          """
34          if 'timeout' in action:
35              remaining = action['timeout']
36          else:
37              remaining = -1  # will never reach 0
38          target_pos = action['go_to']
39          arrived = self.close_enough(target_pos)
40          while not arrived and remaining != 0:
41              self.percept = self.lower.do({"steer":self.steer(target_pos)})
42              remaining -= 1
43              arrived = self.close_enough(target_pos)
44          return {'arrived':arrived}
```

The following method determines how to steer depending on whether the goal is to the right or the left of where the robot is facing.

agentMiddle.py — (continued)

```
46      def steer(self, target_pos):
47          if self.percept['whisker']:
48              self.display(3,'whisker on', self.percept)
49              return "left"
50          else:
51              return self.head_towards(target_pos)
52
53      def head_towards(self, target_pos):
54              """ given a target position, return the action that heads
                    towards that position
55              """
56              gx,gy = target_pos
57              rx,ry = self.percept['rob_pos']
58              goal_dir = math.acos((gx-rx)/math.sqrt((gx-rx)*(gx-rx)
59                                                     +(gy-ry)*(gy-ry)))*180/math.pi
60              if ry>gy:
61                  goal_dir = -goal_dir
62              goal_from_rob = (goal_dir - self.percept['rob_dir']+540)%360-180
63              assert -180 < goal_from_rob <= 180
64              if goal_from_rob > self.straight_angle:
65                  return "left"
66              elif goal_from_rob < -self.straight_angle:
67                  return "right"
68              else:
69                  return "straight"
70
71      def close_enough(self, target_pos):
72          """True when the robot's position is within close_threshold of
                target_pos
73          """
```

```
74          gx,gy = target_pos
75          rx,ry = self.percept['rob_pos']
76          return (gx-rx)**2 + (gy-ry)**2 <= self.close_threshold_squared
```

2.3.3 Top Layer

The top layer treats the middle layer as its environment. Note that the top layer is an environment for us to tell it what to visit.

agentTop.py — Top Layer

```
11  from display import Displayable
12  from agentMiddle import Rob_middle_layer
13  from agents import Agent, Environment
14
15  class Rob_top_layer(Agent, Environment):
16      def __init__(self, lower, world, timeout=200 ):
17          """lower is the lower layer
18          world is the world (which knows where the locations are)
19          timeout is the number of steps the middle layer goes before giving
                up
20          """
21          self.lower = lower
22          self.world = world
23          self.timeout = timeout # number of steps before the middle layer
                should give up
24
25      def do(self,plan):
26          """carry out actions.
27          actions is of the form {'visit':list_of_locations}
28          It visits the locations in turn.
29          """
30          to_do = plan['visit']
31          for loc in to_do:
32              position = self.world.locations[loc]
33              arrived = self.lower.do({'go_to':position,
                    'timeout':self.timeout})
34              self.display(1,"Goal",loc,arrived)
```

2.3.4 World

The world defines the walls and implements tracing.

agentEnv.py — (continued)

```
92  import math
93  from display import Displayable
94  import matplotlib.pyplot as plt
95
96  class World(Environment):
```

```
97      def __init__(self, walls = {}, locations = {},
            plot_size=(-10,120,-10,60)):
98          """walls is a set of line segments
99                where each line segment is of the form ((x0,y0),(x1,y1))
100          locations is a loc:pos dictionary
101             where loc is a named location, and pos is an (x,y) position.
102         """
103         self.walls = walls
104         self.locations = locations
105         self.loc2text = {}
106         self.history = [] # list of (pos, whisker, crashed)
107         # The following control how it is plotted
108         plt.ion()
109         fig, self.ax = plt.subplots()
110         #self.ax.set_aspect('equal')
111         self.ax.axis(plot_size)
112         self.sleep_time = 0.05  # time between actions (for real-time
                plotting)
113         self.draw()
114
115     def do(self, action):
116         """action is {'rob_pos':(x,y), 'whisker':Boolean, 'crashed':Boolean}
117         """
118         self.history.append((action['rob_pos'],action['whisker'],action['crashed']))
119         x,y = action['rob_pos']
120         if action['crashed']:
121             self.display(1, "*Crashed*")
122             self.ax.plot([x],[y],"r*",markersize=20.0)
123         elif action['whisker']:
124             self.ax.plot([x],[y],"ro")
125         else:
126             self.ax.plot([x],[y],"go")
127         plt.draw()
128         plt.pause(self.sleep_time)
129         return {'walls':self.walls}
```

2.3.5 Plotting

The following is used to plot the locations, the walls and (eventually) the movement of the robot. It can either plot the movement if the robot as it is going (with the default *env.plotting* = *True*), or not plot it as it is going (setting *env.plotting* = *False*; in this case the trace can be plotted using *pl.plot_run*()).

agentEnv.py — (continued)

```
131     def draw(self):
132         for wall in self.walls:
133             ((x0,y0),(x1,y1)) = wall
134             self.ax.plot([x0,x1],[y0,y1],"-k",linewidth=3)
135         for loc in self.locations:
```

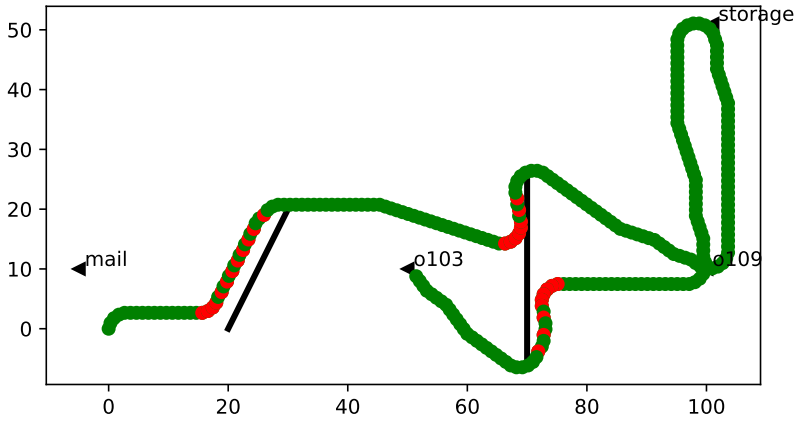


Figure 2.2: A trace of the trajectory of the agent. Red dots correspond to the whisker sensor being on; the green dot to the whisker sensor being off. The agent starts at position $(0,0)$ facing up.

```
136             self.plot_loc(loc)
137
138     def plot_loc(self, loc):
139         (x,y) = self.locations[loc]
140         if loc in self.loc2text:
141             for e in self.loc2text[loc]:
142                 e.remove() # e.set_visible(False)
143         self.loc2text[loc] = (
                self.ax.text(x,y,"*",ha="center",va="center",size=20),
144                         self.ax.text(x+2.0,y+1,loc)) # label above and to
                             the right
```

The following example shows a plot of the agent as it acts in the world. Figure 2.2 shows the result of the commented-out `top.do`

agentTop.py — (continued)

```
36 from agentEnv import Rob_body, World
37
38 def rob_ex():
39     global world, body, middle, top
40     world = World(walls = {((20,0),(30,20)), ((70,-5),(70,25))},
41                   locations = {'mail':(-5,10),
42                                'o103':(50,10),
                                     'o109':(100,10),'storage':(101,51)})
43     body = Rob_body(world)
44     middle = Rob_middle_layer(body)
45     top = Rob_top_layer(middle, world)
46
47 # try:
48 # top.do({'visit':['o109','storage','o109','o103']})
49 # You can directly control the middle layer:
```

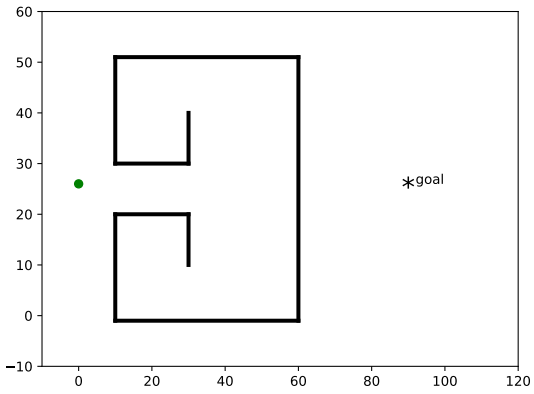


Figure 2.3: Robot trap

```
# middle.do({'go_to':(30,-5), 'timeout':200})
# Can you make it go around in circles?
# Can you make it crash?

if __name__ == "__main__":
    rob_ex()
    print("Try: top.do({'visit':['o109','storage','o109','o103']})")
```

Exercise 2.2 When does the robot go in circles? How could this be recognized and/or avoided?

Exercise 2.3 When does the agent crash? What sensor would avoid that? (Think about the worse configuration of walls.) Design a whisker-like sensor that never crashes (assuming it starts far enough from a wall) and allows the robot to go as close as possible to a wall.

Exercise 2.4 The following implements a robot trap (Figure 2.3). It is called a trap because, once it has hit the wall, it needs to follow the wall, but local features are not enough for it to know when it can head to the goal. Write a controller that can escape the "trap" and get to the goal. Would a beter sensor work? See Exercise 2.4 in the textbook for hints.

agentTop.py — (continued)

```
# Robot Trap for which the current controller cannot escape:
def robot_trap():
    global trap_world, trap_body, trap_middle, trap_top
    trap_world = World({((10, 51), (60, 51)), ((30, 10), (30, 20)),
                        ((10, -1), (10, 20)), ((10, 30), (10, 51)),
                        ((30, 30), (30, 40)), ((10, -1), (60, -1)),
                        ((10, 30), (30, 30)), ((10, 20), (30, 20)),
                        ((60, -1), (60, 51))},
                   locations={'goal':(90,25)})
```

```
    trap_body = Rob_body(trap_world,init_pos=(0,25), init_dir=90)
    trap_middle = Rob_middle_layer(trap_body)
    trap_top = Rob_top_layer(trap_middle, trap_world)

# Robot trap exercise:
# robot_trap()
# trap_body.do({'steer':'straight'})
# trap_top.do({'visit':['goal']})
# What if the goal was further to the right?
```

Plotting for Moving Targets

Exercise 2.5 of Poole and Mackworth [2023] refers to targets that can move. The following implements targets than can be moved using the mouse. To move a target using the mouse, press on the target, move it, and release at the desired location. This can be done while the animation is running.

agentFollowTarget.py — Plotting for moving targets

```
import matplotlib.pyplot as plt
from agentEnv import Rob_body, World
from agentMiddle import Rob_middle_layer
from agentTop import Rob_top_layer

class World_follow(World):
    def __init__(self, walls = {}, locations = {}, epsilon=5):
        """plot the agent in the environment.
        epsilon is the threshold how how close someone needs to click to
            select a location.
        """
        self.epsilon = epsilon
        World.__init__(self, walls, locations)
        self.canvas = self.ax.figure.canvas
        self.canvas.mpl_connect('button_press_event', self.on_press)
        self.canvas.mpl_connect('button_release_event', self.on_release)
        self.canvas.mpl_connect('motion_notify_event', self.on_move)
        self.pressloc = None
        for loc in self.locations:
            self.display(2,f"  loc {loc} at {self.locations[loc]}")

    def on_press(self, event):
        print("press", event)
        self.display(2,'v',end="")
        self.display(2,f"Press at ({event.xdata},{event.ydata}")
        self.pressloc = None
        if event.xdata:
          for loc in self.locations:
            lx,ly = self.locations[loc]
            if abs(event.xdata- lx) <= self.epsilon and abs(event.ydata-
                ly) <= self.epsilon :
```

```
40              self.display(2,f"moving {loc} from ({event.xdata},
                    {event.ydata})" )
41              self.pressloc = loc
42
43      def on_release(self, event):
44          self.display(2,'^',end="")
45          if self.pressloc is not None and event.xdata:
46              self.display(2,f"Placing {self.pressloc} at {(event.xdata,
                    event.ydata)}")
47              self.locations[self.pressloc] = (event.xdata, event.ydata)
48              self.plot_loc(self.pressloc)
49          self.pressloc = None
50
51      def on_move(self, event):
52          if self.pressloc is not None and event.inaxes:
53              self.display(2,'-',end="")
54              self.locations[self.pressloc] = (event.xdata, event.ydata)
55              self.plot_loc(self.pressloc)
56          else:
57              self.display(2,'.',end="")
58
59  def rob_follow():
60      global world, body, middle, top
61      world = World_follow(walls = {((20,0),(30,20)), ((70,-5),(70,25))},
62                   locations = {'mail':(-5,10), 'o103':(50,10),
63                                'o109':(100,10),'storage':(101,51)})
64      body = Rob_body(world)
65      middle = Rob_middle_layer(body)
66      top = Rob_top_layer(middle, world)
67
68  # top.do({'visit':['o109','storage','o109','o103']})
69
70  if __name__ == "__main__":
71      rob_follow()
72      print("Try: top.do({'visit':['o109','storage','o109','o103']})")
```

Exercise 2.5 Do Exercise 2.5 of Poole and Mackworth [2023].

Exercise 2.6 Change the code to also allow walls to move.

Chapter 3

Searching for Solutions

3.1 Representing Search Problems

A search problem consists of:

- a start node
- a *neighbors* function that given a node, returns an enumeration of the arcs from the node
- a specification of a goal in terms of a Boolean function that takes a node and returns true if the node is a goal
- a (optional) heuristic function that, given a node, returns a non-negative real number. The heuristic function defaults to zero.

As far as the searcher is concerned a node can be anything. If multiple-path pruning is used, a node must be `hashable`. In the simple examples, it is a string, but in more complicated examples (in later chapters) it can be a tuple, a frozen set, or a Python object.

In the following code, "`raise NotImplementedError()`" is a way to specify that this is an abstract method that needs to be overridden to define an actual search problem.

searchProblem.py — representations of search problems

```
11 from display import Displayable
12 import matplotlib.pyplot as plt
13 import random
14
15 class Search_problem(Displayable):
16     """A search problem consists of:
```

```
    * a start node
    * a neighbors function that gives the neighbors of a node
    * a specification of a goal
    * a (optional) heuristic function.
    The methods must be overridden to define a search problem."""

    def start_node(self):
        """returns start node"""
        raise NotImplementedError("start_node") # abstract method

    def is_goal(self,node):
        """is True if node is a goal"""
        raise NotImplementedError("is_goal") # abstract method

    def neighbors(self,node):
        """returns a list (or enumeration) of the arcs for the neighbors of
            node"""
        raise NotImplementedError("neighbors") # abstract method

    def heuristic(self,n):
        """Gives the heuristic value of node n.
        Returns 0 if not overridden."""
        return 0
```

The neighbors is a list or enumeration of arcs. A (directed) arc is the pair (`from_node`, `to_node`), but can also contain a non-negative `cost` (which defaults to 1) and can be labeled with an `action`. The action is not used for the search, but is useful for displaying and for plans (sequences of of actions).

searchProblem.py — (continued)

```
class Arc(object):
    """An arc consists of
       a from_node and a to_node node
       a (non-negative) cost
       an (optional) action
    """
    def __init__(self, from_node, to_node, cost=1, action=None):
        self.from_node = from_node
        self.to_node = to_node
        self.cost = cost
        assert cost >= 0, (f"Cost cannot be negative: {self}, cost={cost}")
        self.action = action

    def __repr__(self):
        """string representation of an arc"""
        if self.action:
            return f"{self.from_node} --{self.action}--> {self.to_node}"
        else:
            return f"{self.from_node} --> {self.to_node}"
```

3.1.1 Explicit Representation of Search Graph

The first representation of a search problem is from an explicit graph (as opposed to one that is generated as needed).

An **explicit graph** consists of

- a list or set of nodes
- a list or set of arcs
- a start node
- a list or set of goal nodes
- (optionally) a hmap dictionary that maps a node to a heuristic value for that node. This could conceivably have been part of nodes, but the heuristic value depends on the goals.
- (optionally) a positions dictionary that maps nodes to their x-y position. This is for showing the graph visually.

To define a search problem, you need to define the start node, the goal predicate, the neighbors function and, for some algorithms, a heuristic function.

searchProblem.py — (continued)

```
60 class Search_problem_from_explicit_graph(Search_problem):
61     """A search problem from an explicit graph.
62     """
63 
64     def __init__(self, title, nodes, arcs, start=None, goals=set(), hmap={},
65                  positions=None):
66         """ A search problem consists of:
67         * list or set of nodes
68         * list or set of arcs
69         * start node
70         * list or set of goal nodes
71         * hmap: dictionary that maps each node into its heuristic value.
72         * positions: dictionary that maps each node into its (x,y) position
73         """
74         self.title = title
75         self.neighs = {}
76         self.nodes = nodes
77         for node in nodes:
78             self.neighs[node]=[]
79         self.arcs = arcs
80         for arc in arcs:
81             self.neighs[arc.from_node].append(arc)
82         self.start = start
83         self.goals = goals
84         self.hmap = hmap
85         if positions is None:
```

```
86              self.positions = {node:(random.random(),random.random()) for
                    node in nodes}
87          else:
88              self.positions = positions
89
90      def start_node(self):
91          """returns start node"""
92          return self.start
93
94      def is_goal(self,node):
95          """is True if node is a goal"""
96          return node in self.goals
97
98      def neighbors(self,node):
99          """returns the neighbors of node (a list of arcs)"""
100         return self.neighs[node]
101
102     def heuristic(self,node):
103         """Gives the heuristic value of node n.
104         Returns 0 if not overridden in the hmap."""
105         if node in self.hmap:
106             return self.hmap[node]
107         else:
108             return 0
109
110     def __repr__(self):
111         """returns a string representation of the search problem"""
112         res=""
113         for arc in self.arcs:
114             res += f"{arc}. "
115         return res
```

Graphical Display of a Search Graph

The `show()` method displays the graph, and is used for the figures in this document.

searchProblem.py — (continued)

```
117     def show(self, fontsize=10, node_color='orange', show_costs = True):
118         """Show the graph as a figure
119         """
120         self.fontsize = fontsize
121         self.show_costs = show_costs
122         plt.ion()  # interactive
123         fig, ax = plt.subplots()
124         ax.set_axis_off()
125         ax.set_title(self.title, fontsize=fontsize)
126         self.show_graph(ax, node_color)
127
128     def show_graph(self, ax, node_color='orange'):
```

```
        bbox =
            dict(boxstyle="round4,pad=1.0,rounding_size=0.5",facecolor=node_color)
        for arc in self.arcs:
            self.show_arc(ax, arc)
        for node in self.nodes:
            self.show_node(ax, node, node_color = node_color)

    def show_node(self, ax, node, node_color):
            x,y = self.positions[node]
            ax.text(x,y,node,bbox=dict(boxstyle="round4,pad=1.0,rounding_size=0.5",
                                facecolor=node_color),
                    ha='center',va='center', fontsize=self.fontsize)

    def show_arc(self, ax, arc, arc_color='black', node_color='white'):
            from_pos = self.positions[arc.from_node]
            to_pos = self.positions[arc.to_node]
            ax.annotate(arc.to_node, from_pos, xytext=to_pos,
                        arrowprops={'arrowstyle':'<|-', 'linewidth': 2,
                                    'color':arc_color},
                        bbox=dict(boxstyle="round4,pad=1.0,rounding_size=0.5",
                                          facecolor=node_color),
                               ha='center',va='center',
                               fontsize=self.fontsize)
            # Add costs to middle of arcs:
            if self.show_costs:
                ax.text((from_pos[0]+to_pos[0])/2, (from_pos[1]+to_pos[1])/2,
                         arc.cost, bbox=dict(pad=1,fc='w',ec='w'),
                         ha='center',va='center',fontsize=self.fontsize)
```

3.1.2 Paths

A searcher will return a path from the start node to a goal node. A Python list is not a suitable representation for a path, as many search algorithms consider multiple paths at once, and these paths should share initial parts of the path. If we wanted to do this with Python lists, we would need to keep copying the list, which can be expensive if the list is long. An alternative representation is used here in terms of a recursive data structure that can share subparts.

A path is either:

- a node (representing a path of length 0) or
- an initial path, and an arc at the end, where the `from_node` of the arc is the node at the end of the initial path.

These cases are distinguished in the following code by having `arc=None` if the path has length 0, in which case `initial` is the node of the path. Note that we only use the most basic form of Python's `yield` for enumerations (Section 1.5.3).

searchProblem.py — (continued)

```
class Path(object):
    """A path is either a node or a path followed by an arc"""

    def __init__(self,initial,arc=None):
        """initial is either a node (in which case arc is None) or
        a path (in which case arc is an object of type Arc)"""
        self.initial = initial
        self.arc=arc
        if arc is None:
            self.cost=0
        else:
            self.cost = initial.cost+arc.cost

    def end(self):
        """returns the node at the end of the path"""
        if self.arc is None:
            return self.initial
        else:
            return self.arc.to_node

    def nodes(self):
        """enumerates the nodes of the path from the last element backwards
        """
        current = self
        while current.arc is not None:
            yield current.arc.to_node
            current = current.initial
        yield current.initial

    def initial_nodes(self):
        """enumerates the nodes for the path before the end node.
        This calls nodes() for the initial part of the path.
        """
        if self.arc is not None:
            yield from self.initial.nodes()

    def __repr__(self):
        """returns a string representation of a path"""
        if self.arc is None:
            return str(self.initial)
        elif self.arc.action:
            return f"{self.initial}\n --{self.arc.action}-->
                {self.arc.to_node}"
        else:
            return f"{self.initial} --> {self.arc.to_node}"
```

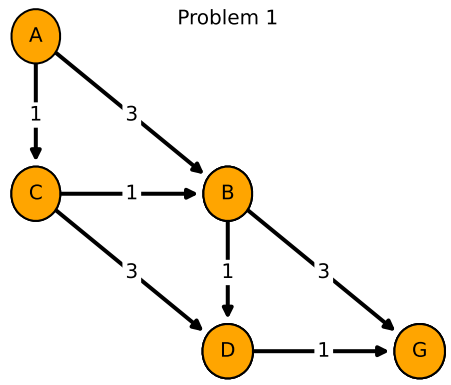


Figure 3.1: problem1

3.1.3 Example Search Problems

The first search problem is one with 5 nodes where the least-cost path is one with many arcs. See Figure 3.1, generated using problem1.show(). Note that this example is used for the unit tests, so the test (in `searchGeneric`) will need to be changed if this is changed.

searchExample.py — Search Examples

```
from searchProblem import Arc, Search_problem_from_explicit_graph,
    Search_problem

problem1 = Search_problem_from_explicit_graph('Problem 1',
    {'A','B','C','D','G'},
    [Arc('A','B',3), Arc('A','C',1), Arc('B','D',1), Arc('B','G',3),
        Arc('C','B',1), Arc('C','D',3), Arc('D','G',1)],
    start = 'A',
    goals = {'G'},
    positions={'A': (0, 1), 'B': (0.5, 0.5), 'C': (0,0.5),
                'D': (0.5,0), 'G': (1,0)})
```

The second search problem is one with 8 nodes where many paths do not lead to the goal. See Figure 3.2.

searchExample.py — (continued)

```
problem2 = Search_problem_from_explicit_graph('Problem 2',
    {'A','B','C','D','E','G','H','J'},
    [Arc('A','B',1), Arc('B','C',3), Arc('B','D',1), Arc('D','E',3),
       Arc('D','G',1), Arc('A','H',3), Arc('H','J',1)],
    start = 'A',
    goals = {'G'},
    positions={'A':(0, 1), 'B':(0, 3/4), 'C':(0,0), 'D':(1/4,3/4),
                'E':(1/4,0), 'G':(2/4,3/4), 'H':(3/4,1), 'J':(3/4,3/4)})
```

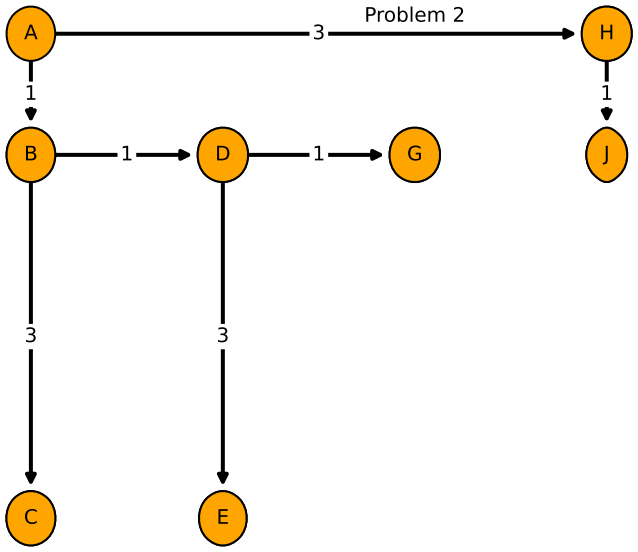


Figure 3.2: problem2

The third search problem is a disconnected graph (contains no arcs), where the start node is a goal node. This is a boundary case to make sure that weird cases work.

searchExample.py — (continued)

```
problem3 = Search_problem_from_explicit_graph('Problem 3',
    {'a','b','c','d','e','g','h','j'},
    [],
    start = 'g',
    goals = {'k','g'})
```

The `simp_delivery_graph` is shown Figure 3.3. This is the same as Figure 3.3 of Poole and Mackworth [2023].

searchExample.py — (continued)

```
simp_delivery_graph = Search_problem_from_explicit_graph("Acyclic Delivery
    Graph",
    {'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'J'},
    [   Arc('A', 'B', 2),
        Arc('A', 'C', 3),
        Arc('A', 'D', 4),
        Arc('B', 'E', 2),
        Arc('B', 'F', 3),
        Arc('C', 'J', 7),
        Arc('D', 'H', 4),
        Arc('F', 'D', 2),
```

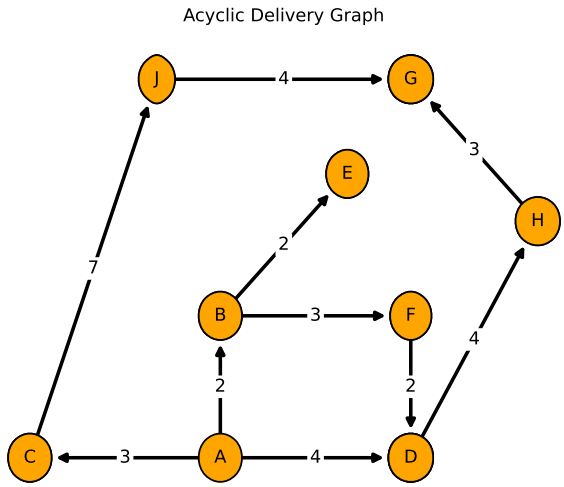


Figure 3.3: simp_delivery_graph.show()

```
47          Arc('H', 'G', 3),
48          Arc('J', 'G', 4)],
49    start = 'A',
50    goals = {'G'},
51    hmap = {
52         'A': 7,
53         'B': 5,
54         'C': 9,
55         'D': 6,
56         'E': 3,
57         'F': 5,
58         'G': 0,
59         'H': 3,
60         'J': 4,
61     },
62     positions = {
63         'A': (0.4,0.1),
64         'B': (0.4,0.4),
65         'C': (0.1,0.1),
66         'D': (0.7,0.1),
67         'E': (0.6,0.7),
68         'F': (0.7,0.4),
69         'G': (0.7,0.9),
70         'H': (0.9,0.6),
71         'J': (0.3,0.9)
72        }
```

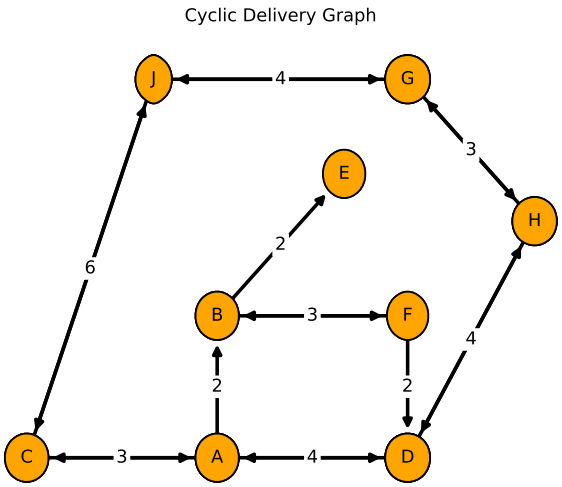


Figure 3.4: `cyclic_simp_delivery_graph.show()`

```
    )
```

`cyclic_simp_delivery_graph` is the graph shown Figure 3.4. This is the graph of Figure 3.10 of [Poole and Mackworth, 2023]. The heuristic values are the same as in `simp_delivery_graph`.

searchExample.py — (continued)

```
cyclic_simp_delivery_graph = Search_problem_from_explicit_graph("Cyclic
    Delivery Graph",
    {'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'J'},
    [   Arc('A', 'B', 2),
        Arc('A', 'C', 3),
        Arc('A', 'D', 4),
        Arc('B', 'E', 2),
        Arc('B', 'F', 3),
        Arc('C', 'A', 3),
        Arc('C', 'J', 6),
        Arc('D', 'A', 4),
        Arc('D', 'H', 4),
        Arc('F', 'B', 3),
        Arc('F', 'D', 2),
        Arc('G', 'H', 3),
        Arc('G', 'J', 4),
        Arc('H', 'D', 4),
        Arc('H', 'G', 3),
        Arc('J', 'C', 6),
        Arc('J', 'G', 4)],
```

```
   start = 'A',
   goals = {'G'},
   hmap = {
        'A': 7,
        'B': 5,
        'C': 9,
        'D': 6,
        'E': 3,
        'F': 5,
        'G': 0,
        'H': 3,
        'J': 4,
    },
    positions = {
        'A': (0.4,0.1),
        'B': (0.4,0.4),
        'C': (0.1,0.1),
        'D': (0.7,0.1),
        'E': (0.6,0.7),
        'F': (0.7,0.4),
        'G': (0.7,0.9),
        'H': (0.9,0.6),
        'J': (0.3,0.9)
        })
```

The next problem is the tree graph shown in Figure 3.5, and is Figure 3.15 in Poole and Mackworth [2023].

searchExample.py — (continued)

```
tree_graph = Search_problem_from_explicit_graph("Tree Graph",
   {'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N',
       'O',
        'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z', 'AA', 'BB',
            'CC',
        'DD', 'EE', 'FF', 'GG', 'HH', 'II', 'JJ', 'KK'},
   [    Arc('A', 'B', 1),
        Arc('A', 'C', 1),
        Arc('B', 'D', 1),
        Arc('B', 'E', 1),
        Arc('C', 'F', 1),
        Arc('C', 'G', 1),
        Arc('D', 'H', 1),
        Arc('D', 'I', 1),
        Arc('E', 'J', 1),
        Arc('E', 'K', 1),
        Arc('F', 'L', 1),
        Arc('G', 'M', 1),
        Arc('G', 'N', 1),
        Arc('H', 'O', 1),
        Arc('H', 'P', 1),
        Arc('J', 'Q', 1),
```

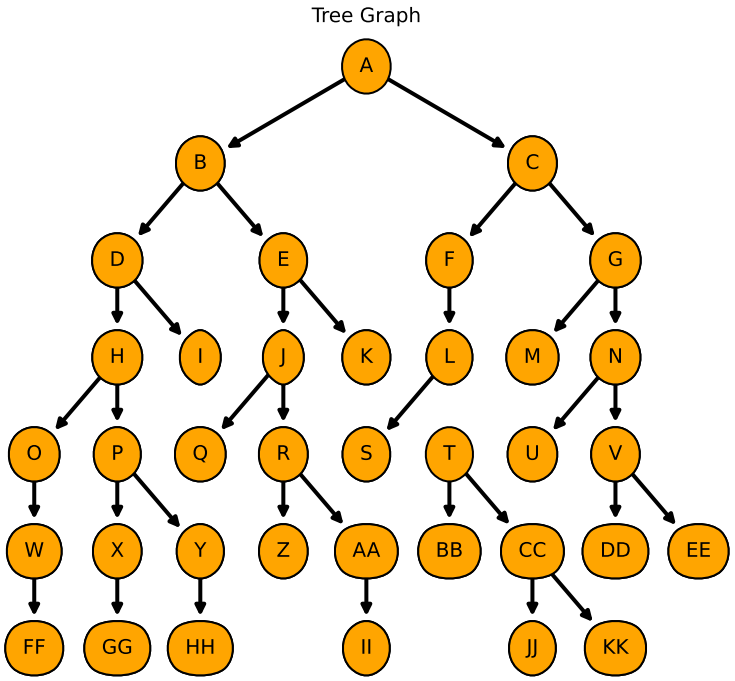


Figure 3.5: `tree_graph.show(show_costs = False)`

```
        Arc('J', 'R', 1),
        Arc('L', 'S', 1),
        Arc('L', 'T', 1),
        Arc('N', 'U', 1),
        Arc('N', 'V', 1),
        Arc('O', 'W', 1),
        Arc('P', 'X', 1),
        Arc('P', 'Y', 1),
        Arc('R', 'Z', 1),
        Arc('R', 'AA', 1),
        Arc('T', 'BB', 1),
        Arc('T', 'CC', 1),
        Arc('V', 'DD', 1),
        Arc('V', 'EE', 1),
        Arc('W', 'FF', 1),
        Arc('X', 'GG', 1),
        Arc('Y', 'HH', 1),
        Arc('AA', 'II', 1),
```

```
        Arc('CC', 'JJ', 1),
        Arc('CC', 'KK', 1)
    ],
   start = 'A',
   goals = {'K', 'M', 'T', 'X', 'Z', 'HH'},
    positions = {
        'A': (0.5,0.95),
        'B': (0.3,0.8),
        'C': (0.7,0.8),
        'D': (0.2,0.65),
        'E': (0.4,0.65),
        'F': (0.6,0.65),
        'G': (0.8,0.65),
        'H': (0.2,0.5),
        'I': (0.3,0.5),
        'J': (0.4,0.5),
        'K': (0.5,0.5),
        'L': (0.6,0.5),
        'M': (0.7,0.5),
        'N': (0.8,0.5),
        'O': (0.1,0.35),
        'P': (0.2,0.35),
        'Q': (0.3,0.35),
        'R': (0.4,0.35),
        'S': (0.5,0.35),
        'T': (0.6,0.35),
        'U': (0.7,0.35),
        'V': (0.8,0.35),
        'W': (0.1,0.2),
        'X': (0.2,0.2),
        'Y': (0.3,0.2),
        'Z': (0.4,0.2),
        'AA': (0.5,0.2),
        'BB': (0.6,0.2),
        'CC': (0.7,0.2),
        'DD': (0.8,0.2),
        'EE': (0.9,0.2),
        'FF': (0.1,0.05),
        'GG': (0.2,0.05),
        'HH': (0.3,0.05),
        'II': (0.5,0.05),
        'JJ': (0.7,0.05),
        'KK': (0.8,0.05)
        }
    )

# tree_graph.show(show_costs = False)
```

3.2 Generic Searcher and Variants

To run the search demos, in folder "aipython", load "searchGeneric.py" , using e.g., ipython -i searchGeneric.py, and copy and paste the example queries at the bottom of that file.

3.2.1 Searcher

A *Searcher* for a problem can be asked repeatedly for the next path. To solve a search problem, construct a *Searcher* object for the problem and then repeatedly ask for the next path using *search*. If there are no more paths, *None* is returned.

searchGeneric.py — Generic Searcher, including depth-first and A*

```
11 from display import Displayable
12
13 class Searcher(Displayable):
14     """returns a searcher for a problem.
15     Paths can be found by repeatedly calling search().
16     This does depth-first search unless overridden
17     """
18     def __init__(self, problem):
19         """creates a searcher from a problem
20         """
21         self.problem = problem
22         self.initialize_frontier()
23         self.num_expanded = 0
24         self.add_to_frontier(Path(problem.start_node()))
25         super().__init__()
26
27     def initialize_frontier(self):
28         self.frontier = []
29
30     def empty_frontier(self):
31         return self.frontier == []
32
33     def add_to_frontier(self,path):
34         self.frontier.append(path)
35
36     def search(self):
37         """returns (next) path from the problem's start node
38         to a goal node.
39         Returns None if no path exists.
40         """
41         while not self.empty_frontier():
42             self.path = self.frontier.pop()
43             self.num_expanded += 1
44             if self.problem.is_goal(self.path.end()): # solution found
45                 self.solution = self.path # store the solution found
```

```
46              self.display(1, f"Solution: {self.path} (cost:
                    {self.path.cost})\n",
47                  self.num_expanded, "paths have been expanded and",
48                         len(self.frontier), "paths remain in the
                               frontier")
49              return self.path
50          else:
51              self.display(4,f"Expanding: {self.path} (cost:
                    {self.path.cost})")
52              neighs = self.problem.neighbors(self.path.end())
53              self.display(2,f"Expanding: {self.path} with neighbors
                    {neighs}")
54              for arc in reversed(list(neighs)):
55                  self.add_to_frontier(Path(self.path,arc))
56              self.display(3, f"New frontier: {[p.end() for p in
                    self.frontier]}")
57
58      self.display(0,"No (more) solutions. Total of",
59                   self.num_expanded,"paths expanded.")
```

Note that this reverses the neighbors so that it implements depth-first search in an intuitive manner (expanding the first neighbor first). The call to *list* is for the case when the neighbors are generated (and not already in a list). Reversing the neighbors might not be required for other methods. The calls to *reversed* and *list* can be removed, and the algorithm still implements depth-first search.

To use depth-first search to find multiple paths for `problem1` and `simp_delivery_graph`, copy and paste the following into Python's read-evaluate-print loop; keep finding next solutions until there are no more:

searchGeneric.py — (continued)

```
61 # Depth-first search for problem1:
62 # searcher1 = Searcher(searchExample.problem1)
63 # searcher1.search() # find first solution
64 # searcher1.search() # find next solution (repeat until no solutions)
65
66 # Depth-first search for simple delivery graph:
67 # searcher_sdg = Searcher(searchExample.simp_delivery_graph)
68 # searcher_sdg.search() # find first or next solution
```

Exercise 3.1 Implement breadth-first search. Only *add_to_frontier* and/or *pop* need to be modified to implement a first-in first-out queue.

3.2.2 GUI for Tracing Search

[This GUI implements most of the functionality of the solve model of the now-discontinued AISpace.org search app.]

Figure 3.6 shows the GUI that can be used to step through search algorithms. Here the path $A \rightarrow B$ is being expanded, and the neighbors are E and F. The other nodes at the end of paths of the frontier are C and D. Thus the

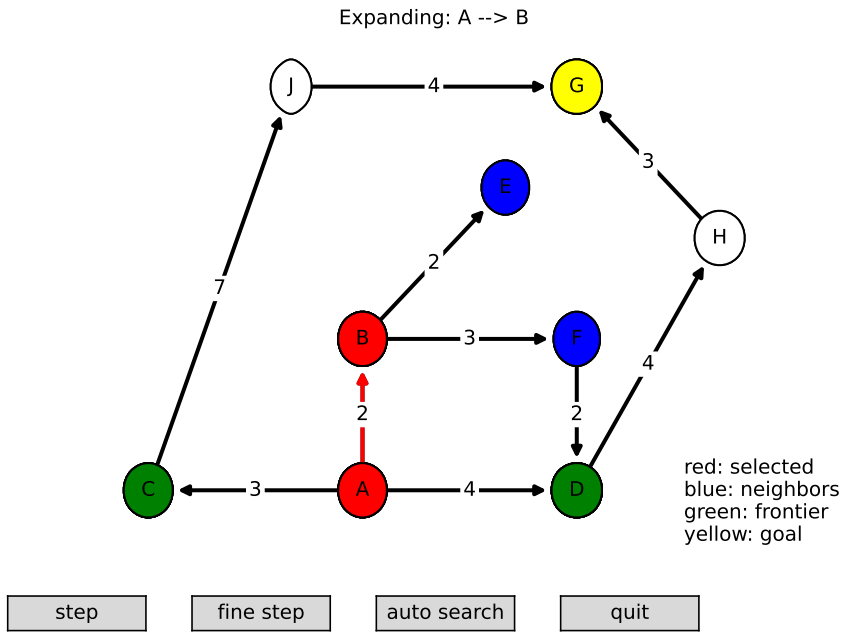


Figure 3.6: `SearcherGUI(Searcher, simp_delivery_graph)`

frontier contains paths to C and D, used to also contain $A \rightarrow B$, and now will contain $A \rightarrow B \rightarrow E$ and $A \rightarrow B \rightarrow F$.

`SearcherGUI` takes a search class and a problem, and lets one explore the search space after calling `go()`. A GUI can only be used for one search; at the end of the search the loop ends and the buttons no longer work.

This is implemented by redefining `display`. The search algorithms don't need to be modified. If you modify them (or create your own), you just have to be careful to use the appropriate number for the display. The first argument to display has the following meanings:

1. a solution has been found
2. what is shown for a "step" on a GUI; here it is assumed to be the path, the neighbors of the end of the path, and the other nodes at the end of paths on the frontier
3. (shown with "fine step" but not with "step") the frontier and the path selected
4. (shown with "fine step" but not with "step") the frontier.

It is also useful to look at the Python console, as the display information is printed there.

searchGUI.py — GUI for search

```
import matplotlib.pyplot as plt
from matplotlib.widgets import Button
import time

class SearcherGUI(object):
    def __init__(self, SearchClass, problem,
                 fontsize=10,
                 colors = {'selected':'red', 'neighbors':'blue',
                     'frontier':'green', 'goal':'yellow'},
                 show_costs = True):
        self.problem = problem
        self.searcher = SearchClass(problem)
        self.problem.fontsize = fontsize
        self.colors = colors
        self.problem.show_costs = show_costs
        self.quitting = False

        fig, self.ax = plt.subplots()
        plt.ion()  # interactive
        self.ax.set_axis_off()
        plt.subplots_adjust(bottom=0.15)
        step_butt = Button(fig.add_axes([0.1,0.02,0.2,0.05]), "step")
        step_butt.on_clicked(self.step)
        fine_butt = Button(fig.add_axes([0.4,0.02,0.2,0.05]), "fine step")
        fine_butt.on_clicked(self.finestep)
        auto_butt = Button(fig.add_axes([0.7,0.02,0.2,0.05]), "auto search")
        auto_butt.on_clicked(self.auto)
        fig.canvas.mpl_connect('close_event', self.window_closed)
        self.ax.text(0.85,0, '\n'.join(self.colors[a]+": "+a
                                       for a in self.colors))
        self.problem.show_graph(self.ax, node_color='white')
        self.problem.show_node(self.ax, self.problem.start,
                               self.colors['frontier'])
        for node in self.problem.nodes:
            if self.problem.is_goal(node):
                self.problem.show_node(self.ax, node,self.colors['goal'])
        plt.show()
        self.click = 7 # bigger than any display!
        self.searcher.display = self.display
        try:
            while self.searcher.frontier:
                path = self.searcher.search()
        except ExitToPython:
            print("GUI closed")
        else:
            print("No more solutions")

    def display(self, level, *args, **nargs):
        if self.quitting:
```

```
            raise ExitToPython()
        if level <= self.click: #step
            print(*args, **nargs)
            self.ax.set_title(f"Expanding: {self.searcher.path}",
                              fontsize=self.problem.fontsize)
            if level == 1:
                self.show_frontier(self.colors['frontier'])
                self.show_path(self.colors['selected'])
                self.ax.set_title(f"Solution Found: {self.searcher.path}",
                                  fontsize=self.problem.fontsize)
            elif level == 2: # what should be shown if node in multiple?
                self.show_frontier(self.colors['frontier'])
                self.show_path(self.colors['selected'])
                self.show_neighbors(self.colors['neighbors'])
            elif level == 3:
                self.show_frontier(self.colors['frontier'])
                self.show_path(self.colors['selected'])
            elif level == 4:
                self.show_frontier(self.colors['frontier'])


            # wait for a button click
            self.click = 0
            plt.draw()
            while self.click == 0 and not self.quitting:
                plt.pause(0.1)
            if self.quitting:
                raise ExitToPython()
            # undo coloring:
            self.ax.set_title("")
            self.show_frontier('white')
            self.show_neighbors('white')
            path_show = self.searcher.path
            while path_show.arc:
                self.problem.show_arc(self.ax, path_show.arc, 'black')
                self.problem.show_node(self.ax, path_show.end(), 'white')
                path_show = path_show.initial
            self.problem.show_node(self.ax, path_show.end(), 'white')
            if self.problem.is_goal(self.searcher.path.end()):
                self.problem.show_node(self.ax, self.searcher.path.end(),
                                       self.colors['goal'])
            plt.draw()

    def show_frontier(self, color):
        for path in self.searcher.frontier:
            self.problem.show_node(self.ax, path.end(), color)

    def show_path(self, color):
        """color selected path"""
        path_show = self.searcher.path
```

```
        while path_show.arc:
                self.problem.show_arc(self.ax, path_show.arc, color)
                self.problem.show_node(self.ax, path_show.end(), color)
                path_show = path_show.initial
        self.problem.show_node(self.ax, path_show.end(), color)

    def show_neighbors(self, color):
        for neigh in self.problem.neighbors(self.searcher.path.end()):
            self.problem.show_node(self.ax, neigh.to_node, color)

    def auto(self, event):
        self.click = 1
    def step(self,event):
        self.click = 2
    def finestep(self, event):
        self.click = 3
    def window_closed(self, event):
        self.quitting = True

class ExitToPython(Exception):
    pass
```

searchGUI.py — (continued)

```
from searchGeneric import Searcher, AStarSearcher
from searchMPP import SearcherMPP
import searchExample
from searchBranchAndBound import DF_branch_and_bound

# to demonstrate depth-first search:
# sdfs = SearcherGUI(Searcher, searchExample.tree_graph)

# delivery graph examples:
# sh = SearcherGUI(Searcher, searchExample.simp_delivery_graph)
# sha = SearcherGUI(AStarSearcher, searchExample.simp_delivery_graph)
# shac = SearcherGUI(AStarSearcher,
    searchExample.cyclic_simp_delivery_graph)
# shm = SearcherGUI(SearcherMPP, searchExample.cyclic_simp_delivery_graph)
# shb = SearcherGUI(DF_branch_and_bound, searchExample.simp_delivery_graph)

# The following is AI:FCA figure 3.15, and is useful to show branch&bound:
# shbt = SearcherGUI(DF_branch_and_bound, searchExample.tree_graph)

if __name__ == "__main__":
    print("Try e.g.: SearcherGUI(Searcher,
        searchExample.simp_delivery_graph)")
```

3.2.3 Frontier as a Priority Queue

In many of the search algorithms, such as A^* and other best-first searchers, the frontier is implemented as a priority queue. The following code uses the Python's built-in priority queue implementations, `heapq`.

Following the lead of the Python documentation, `https://docs.python.org/3/library/heapq.html`, a frontier is a list of triples. The first element of each triple is the value to be minimized. The second element is a unique index which specifies the order that the elements were added to the queue, and the third element is the path that is on the queue. The use of the unique index ensures that the priority queue implementation does not compare paths; whether one path is less than another is not defined. It also lets us control what sort of search (e.g., depth-first or breadth-first) occurs when the value to be minimized does not give a unique next path.

The variable *frontier_index* is the total number of elements of the frontier that have been created. As well as being used as the unique index, it is useful for statistics, particularly in conjunction with the current size of the frontier.

searchGeneric.py — (continued)

```
70 import heapq      # part of the Python standard library
71 from searchProblem import Path
72
73 class FrontierPQ(object):
74     """A frontier consists of a priority queue (heap), frontierpq, of
75         (value, index, path) triples, where
76     * value is the value we want to minimize (e.g., path cost + h).
77     * index is a unique index for each element
78     * path is the path on the queue
79     Note that the priority queue always returns the smallest element.
80     """
81
82     def __init__(self):
83         """constructs the frontier, initially an empty priority queue
84         """
85         self.frontier_index = 0 # the number of items added to the frontier
86         self.frontierpq = [] # the frontier priority queue
87
88     def empty(self):
89         """is True if the priority queue is empty"""
90         return self.frontierpq == []
91
92     def add(self, path, value):
93         """add a path to the priority queue
94         value is the value to be minimized"""
95         self.frontier_index += 1 # get a new unique index
96         heapq.heappush(self.frontierpq,(value, -self.frontier_index, path))
97
98     def pop(self):
99         """returns and removes the path of the frontier with minimum value.
```

```
        """
        (_,_,path) = heapq.heappop(self.frontierpq)
        return path
```

The following methods are used for finding and printing information about the frontier.

searchGeneric.py — (continued)

```
    def count(self,val):
        """returns the number of elements of the frontier with value=val"""
        return sum(1 for e in self.frontierpq if e[0]==val)

    def __repr__(self):
        """string representation of the frontier"""
        return str([(n,c,str(p)) for (n,c,p) in self.frontierpq])

    def __len__(self):
        """length of the frontier"""
        return len(self.frontierpq)

    def __iter__(self):
        """iterate through the paths in the frontier"""
        for (_,_,path) in self.frontierpq:
            yield path
```

3.2.4 A^* Search

For an A^* **Search** the frontier is implemented using the FrontierPQ class.

searchGeneric.py — (continued)

```
class AStarSearcher(Searcher):
    """returns a searcher for a problem.
    Paths can be found by repeatedly calling search().
    """

    def __init__(self, problem):
        super().__init__(problem)

    def initialize_frontier(self):
        self.frontier = FrontierPQ()

    def empty_frontier(self):
        return self.frontier.empty()

    def add_to_frontier(self,path):
        """add path to the frontier with the appropriate cost"""
        value = path.cost+self.problem.heuristic(path.end())
        self.frontier.add(path, value)
```

Code should always be tested. The following provides a simple **unit test**, using `problem1` as the default problem.

searchGeneric.py — (continued)

```
140 import searchExample
141
142 def test(SearchClass, problem=searchExample.problem1,
        solutions=[['G','D','B','C','A']] ):
143     """Unit test for aipython searching algorithms.
144     SearchClass is a class that takes a problem and implements search()
145     problem is a search problem
146     solutions is a list of optimal solutions
147     """
148     print("Testing problem 1:")
149     schr1 = SearchClass(problem)
150     path1 = schr1.search()
151     print("Path found:",path1)
152     assert path1 is not None, "No path is found in problem1"
153     assert list(path1.nodes()) in solutions, "Shortest path not found in
            problem1"
154     print("Passed unit test")
155
156 if __name__ == "__main__":
157     #test(Searcher)    # what needs to be changed to make this succeed?
158     test(AStarSearcher)
159
160 # example queries:
161 # searcher1 = Searcher(searchExample.simp_delivery_graph) # DFS
162 # searcher1.search() # find first path
163 # searcher1.search() # find next path
164 # searcher2 = AStarSearcher(searchExample.simp_delivery_graph) # A*
165 # searcher2.search() # find first path
166 # searcher2.search() # find next path
167 # searcher3 = Searcher(searchExample.cyclic_simp_delivery_graph) # DFS
168 # searcher3.search() # find first path with DFS. What do you expect to
        happen?
169 # searcher4 = AStarSearcher(searchExample.cyclic_simp_delivery_graph) # A*
170 # searcher4.search() # find first path
171
172 # To use the GUI for A* search do the following
173 # python -i searchGUI.py
174 # SearcherGUI(AStarSearcher, searchExample.simp_delivery_graph)
175 # SearcherGUI(AStarSearcher, searchExample.cyclic_simp_delivery_graph)
```

Exercise 3.2 Change the code so that it implements (i) best-first search and (ii) lowest-cost-first search. For each of these methods compare it to A^* in terms of the number of paths expanded, and the path found.

Exercise 3.3 The searcher acts like a Python iterator, in that it returns one value (here a path) and then returns other values (paths) on demand, but does not implement the iterator interface. Change the code so it implements the iterator interface. What does this enable us to do?

3.2.5 Multiple Path Pruning

> To run the multiple-path pruning demo, in folder "aipython", load "searchMPP.py" , using e.g., ipython -i searchMPP.py, and copy and paste the example queries at the bottom of that file.

The following implements A^* with multiple-path pruning. It overrides *search*() in *Searcher*.

searchMPP.py — Searcher with multiple-path pruning

```
from searchGeneric import AStarSearcher
from searchProblem import Path

class SearcherMPP(AStarSearcher):
    """returns a searcher for a problem.
    Paths can be found by repeatedly calling search().
    """
    def __init__(self, problem):
        super().__init__(problem)
        self.explored = set()

    def search(self):
        """returns next path from an element of problem's start nodes
        to a goal node.
        Returns None if no path exists.
        """
        while not self.empty_frontier():
            self.path = self.frontier.pop()
            if self.path.end() not in self.explored:
                self.explored.add(self.path.end())
                self.num_expanded += 1
                if self.problem.is_goal(self.path.end()):
                    self.solution = self.path # store the solution found
                    self.display(1, f"Solution: {self.path} (cost:
                        {self.path.cost})\n",
                    self.num_expanded, "paths have been expanded and",
                           len(self.frontier), "paths remain in the
                               frontier")
                    return self.path
                else:
                    self.display(4,f"Expanding: {self.path} (cost:
                        {self.path.cost})")
                    neighs = self.problem.neighbors(self.path.end())
                    self.display(2,f"Expanding: {self.path} with neighbors
                        {neighs}")
                    for arc in neighs:
                        self.add_to_frontier(Path(self.path,arc))
                    self.display(3, f"New frontier: {[p.end() for p in
                        self.frontier]}")
        self.display(0,"No (more) solutions. Total of",
```

```
                    self.num_expanded,"paths expanded.")

from searchGeneric import test
if __name__ == "__main__":
    test(SearcherMPP)

import searchExample
# searcherMPPcdp = SearcherMPP(searchExample.cyclic_simp_delivery_graph)
# searcherMPPcdp.search() # find first path

# To use the GUI for SearcherMPP do
# python -i searchGUI.py
# import searchMPP
# SearcherGUI(searchMPP.SearcherMPP,
    searchExample.cyclic_simp_delivery_graph)
```

Exercise 3.4 Chris was very puzzled as to why there was a minus ("$-$") in the second element of the tuple added to the heap in the `add` method in `FrontierPQ` in `searchGeneric.py`.

Sam suggested the following example would demonstrate the importance of the minus. Consider an infinite integer grid, where the states are pairs of integers, the start is (0,0), and the goal is (10,10). The neighbors of (i, j) are $(i+1, j)$ and $(i, j+1)$. Consider the heuristic function $h((i, j)) = |10-i|+|10-j|$. Sam suggested you compare how many paths are expanded with the minus and without the minus. `searchGrid` is a representation of Sam's graph. If something takes too long, you might consider changing the size.

searchGrid.py — A grid problem to demonstrate A*

```
from searchProblem import Search_problem, Arc

class GridProblem(Search_problem):
    """a node is a pair (x,y)"""
    def __init__(self, size=10):
        self.size = size

    def start_node(self):
        """returns the start node"""
        return (0,0)

    def is_goal(self,node):
        """returns True when node is a goal node"""
        return node == (self.size,self.size)

    def neighbors(self,node):
        """returns a list of the neighbors of node"""
        (x,y) = node
        return [Arc(node,(x+1,y)), Arc(node,(x,y+1))]

    def heuristic(self,node):
        (x,y) = node
```

```
33          return abs(x-self.size)+abs(y-self.size)
34
35  class GridProblemNH(GridProblem):
36      """Grid problem with a heuristic of 0"""
37      def heuristic(self,node):
38          return 0
39
40  from searchGeneric import Searcher, AStarSearcher
41  from searchMPP import SearcherMPP
42  from searchBranchAndBound import DF_branch_and_bound
43
44  def testGrid(size = 10):
45      print("\nWith MPP")
46      gridsearchermpp = SearcherMPP(GridProblem(size))
47      print(gridsearchermpp.search())
48      print("\nWithout MPP")
49      gridsearchera = AStarSearcher(GridProblem(size))
50      print(gridsearchera.search())
51      print("\nWith MPP and a heuristic = 0 (Dijkstra's algorithm)")
52      gridsearchermppnh = SearcherMPP(GridProblemNH(size))
53      print(gridsearchermppnh.search())
```

Explain to Chris what the minus does and why it is there. Give evidence for your claims. It might be useful to refer to other search strategies in your explanation. As part of your explanation, explain what is special about Sam's example.

Exercise 3.5 Implement a searcher that implements cycle pruning instead of multiple-path pruning. You need to decide whether to check for cycles when paths are added to the frontier or when they are removed. (Hint: either method can be implemented by only changing one or two lines in SearcherMPP. Hint: there is a cycle if `path.end() in path.initial_nodes()`) Compare no pruning, multiple path pruning and cycle pruning for the cyclic delivery problem. Which works better in terms of number of paths expanded, computational time or space?

3.3 Branch-and-bound Search

To run the demo, in folder "`aipython`", load "`searchBranchAndBound.py`", and copy and paste the example queries at the bottom of that file.

Depth-first search methods do not need a priority queue, but can use a list as a stack. In this implementation of branch-and-bound search, we call *search* to find an optimal solution with cost less than bound. This uses depth-first search to find a path to a goal that extends *path* with cost less than the bound. Once a path to a goal has been found, that path is remembered as the *best_path*, the bound is reduced, and the search continues.

searchBranchAndBound.py — Branch and Bound Search

```
11  from searchProblem import Path
```

```
from searchGeneric import Searcher
from display import Displayable

class DF_branch_and_bound(Searcher):
    """returns a branch and bound searcher for a problem.
    An optimal path with cost less than bound can be found by calling search()
    """
    def __init__(self, problem, bound=float("inf")):
        """creates a searcher than can be used with search() to find an optimal path.
        bound gives the initial bound. By default this is infinite - meaning there
        is no initial pruning due to depth bound
        """
        super().__init__(problem)
        self.best_path = None
        self.bound = bound

    def search(self):
        """returns an optimal solution to a problem with cost less than bound.
        returns None if there is no solution with cost less than bound."""
        self.frontier = [Path(self.problem.start_node())]
        self.num_expanded = 0
        while self.frontier:
            self.path = self.frontier.pop()
            if self.path.cost+self.problem.heuristic(self.path.end()) < self.bound:
                # if self.path.end() not in self.path.initial_nodes(): # for cycle pruning
                self.display(2,"Expanding:",self.path,"cost:",self.path.cost)
                self.num_expanded += 1
                if self.problem.is_goal(self.path.end()):
                    self.best_path = self.path
                    self.bound = self.path.cost
                    self.display(1,"New best path:",self.path,"cost:",self.path.cost)
                else:
                    neighs = self.problem.neighbors(self.path.end())
                    self.display(4,"Neighbors are", neighs)
                    for arc in reversed(list(neighs)):
                        self.add_to_frontier(Path(self.path, arc))
                    self.display(3, f"New frontier: {[p.end() for p in self.frontier]}")
        self.path = self.best_path
        self.solution = self.best_path
        self.display(1,f"Optimal solution is {self.best_path}." if self.best_path
                       else "No solution found.",
```

```
                        f"Number of paths expanded: {self.num_expanded}.")
        return self.best_path
```

Note that this code used *reversed* in order to expand the neighbors of a node in the left-to-right order one might expect. It does this because *pop*() removes the rightmost element of the list. The call to *list* is there because *reversed* only works on lists and tuples, but the neighbors can be generated.

Here is a unit test and some queries:

searchBranchAndBound.py — (continued)

```
from searchGeneric import test
if __name__ == "__main__":
    test(DF_branch_and_bound)

# Example queries:
import searchExample
# searcherb1 = DF_branch_and_bound(searchExample.simp_delivery_graph)
# searcherb1.search()     # find optimal path
# searcherb2 =
    DF_branch_and_bound(searchExample.cyclic_simp_delivery_graph,
    bound=100)
# searcherb2.search()     # find optimal path

# to use the GUI do:
# ipython -i searchGUI.py
# import searchBranchAndBound
# SearcherGUI(searchBranchAndBound.DF_branch_and_bound,
    searchExample.simp_delivery_graph)
# SearcherGUI(searchBranchAndBound.DF_branch_and_bound,
    searchExample.cyclic_simp_delivery_graph)
```

Exercise 3.6 In searcherb2, in the code above, what happens if the bound is smaller, say 10? What if it is larger, say 1000?

Exercise 3.7 Implement a branch-and-bound search using recursion. Hint: you don't need an explicit frontier, but can do a recursive call for the children.

Exercise 3.8 Add loop detection to branch-and-bound search.

Exercise 3.9 After the branch-and-bound search found a solution, Sam ran search again, and noticed a different count. Sam hypothesized that this count was related to the number of nodes that an $A*$ search would use (either expand or be added to the frontier). Or maybe, Sam thought, the count for a number of nodes when the bound is slightly above the optimal path case is related to how A^* would work. Is there a relationship between these counts? Are there different things that it could count so they are related? Try to find the most specific statement that is true, and explain why it is true.

To test the hypothesis, Sam wrote the following code, but isn't sure it is helpful:

searchTest.py — code that may be useful to compare A* and branch-and-bound

```
from searchGeneric import Searcher, AStarSearcher
```

```
from searchBranchAndBound import DF_branch_and_bound
from searchMPP import SearcherMPP

DF_branch_and_bound.max_display_level = 1
Searcher.max_display_level = 1

def run(problem,name):
    print("\n\n*******",name)

    print("\nA*:")
    asearcher = AStarSearcher(problem)
    print("Path found:",asearcher.search()," cost=",asearcher.solution.cost)
    print("there are",asearcher.frontier.count(asearcher.solution.cost),
          "elements remaining on the queue with
              f-value=",asearcher.solution.cost)

    print("\nA* with MPP:"),
    msearcher = SearcherMPP(problem)
    print("Path found:",msearcher.search()," cost=",msearcher.solution.cost)
    print("there are",msearcher.frontier.count(msearcher.solution.cost),
          "elements remaining on the queue with
              f-value=",msearcher.solution.cost)

    bound = asearcher.solution.cost*1.00001
    print("\nBranch and bound (with too-good initial bound of", bound,")")
    tbb = DF_branch_and_bound(problem,bound) # cheating!!!!
    print("Path found:",tbb.search()," cost=",tbb.solution.cost)
    print("Rerunning B&B")
    print("Path found:",tbb.search())

    bbound = asearcher.solution.cost*10+10
    print("\nBranch and bound (with not-very-good initial bound of",
        bbound, ")")
    tbb2 = DF_branch_and_bound(problem,bbound)
    print("Path found:",tbb2.search()," cost=",tbb2.solution.cost)
    print("Rerunning B&B")
    print("Path found:",tbb2.search())

    print("\nDepth-first search: (Use ^C if it goes on forever)")
    tsearcher = Searcher(problem)
    print("Path found:",tsearcher.search()," cost=",tsearcher.solution.cost)


import searchExample
from searchTest import run
if __name__ == "__main__":
    run(searchExample.problem1,"Problem 1")
#   run(searchExample.simp_delivery_graph,"Acyclic Delivery")
#   run(searchExample.cyclic_simp_delivery_graph,"Cyclic Delivery")
# also test graphs with cycles, and graphs with multiple least-cost paths
```

Chapter 4

Reasoning with Constraints

4.1 Constraint Satisfaction Problems

4.1.1 Variables

A **variable** consists of a name, a domain and an optional (x,y) position (for displaying). The domain of a variable is a list or a tuple, as the ordering matters for some algorithms.

variable.py — Representations of a variable in CSPs and probabilistic models

```
import random

class Variable(object):
    """A random variable.
    name (string) - name of the variable
    domain (list) - a list of the values for the variable.
    an (x,y) position for displaying
    """

    def __init__(self, name, domain, position=None):
        """Variable
        name a string
        domain a list of printable values
        position of form (x,y) where 0 <= x <= 1, 0 <= y <= 1
        """
        self.name = name # string
        self.domain = domain # list of values
        self.position = position if position else (random.random(),
            random.random())
        self.size = len(domain)

    def __str__(self):
```

```
32          return self.name
33
34      def __repr__(self):
35          return self.name # f"Variable({self.name})"
```

4.1.2 Constraints

A **constraint** consists of:

- A tuple (or list) of variables called the **scope**.
- A **condition**, a Boolean function that takes the same number of arguments as there are variables in the scope.
- An name (for displaying)
- An optional (x, y) position. The mean of the positions of the variables in the scope is used, if not specified.

cspProblem.py — Representations of a Constraint Satisfaction Problem

```
11 from variable import Variable
12
13 # for showing csps:
14 import matplotlib.pyplot as plt
15 import matplotlib.lines as lines
16
17 class Constraint(object):
18     """A Constraint consists of
19     * scope: a tuple or list of variables
20     * condition: a Boolean function that can applied to a tuple of values
           for variables in scope
21     * string: a string for printing the constraint
22     """
23     def __init__(self, scope, condition, string=None, position=None):
24         self.scope = scope
25         self.condition = condition
26         self.string = string
27         self.position = position
28
29     def __repr__(self):
30         return self.string
```

An **assignment** is a *variable*:*value* dictionary.
If con is a constraint:

- `con.can_evaluate(assignment)` is True when the constraint can be evaluated in the assignment. Generally this is true when all variables in the scope of the constraint are assigned in the assignment. [There are cases where it could be true when not all variables are assigned, such as if the constraint was "if x then y else z", but that it not implemented here.]

- `con.holds(assignment)` returns True or False depending on whether the condition is true or false for that assignment. The assignment `assignment` must assign a value to every variable in the scope of the constraint `con` (and could also assign values to other variables); `con.holds` gives an error if not all variables in the scope of `con` are assigned in the assignment. It ignores variables in `assignment` that are not in the scope of the constraint.

In Python, the $*$ notation is used for unpacking a tuple. For example, $F(*(1,2,3))$ is the same as $F(1,2,3)$. So if t has value $(1,2,3)$, then $F(*t)$ is the same as $F(1,2,3)$.

cspProblem.py — (continued)

```
32     def can_evaluate(self, assignment):
33         """
34         assignment is a variable:value dictionary
35         returns True if the constraint can be evaluated given assignment
36         """
37         return all(v in assignment for v in self.scope)
38 
39     def holds(self,assignment):
40         """returns the value of Constraint con evaluated in assignment.
41 
42         precondition: all variables are assigned in assignment, ie
               self.can_evaluate(assignment) is true
43         """
44         return self.condition(*tuple(assignment[v] for v in self.scope))
```

4.1.3 CSPs

A constraint satisfaction problem (CSP) requires:

- `title`: a string title
- `variables`: a list or set of variables
- `constraints`: a set or list of constraints.

Other properties are inferred from these:

- `var_to_const` is a mapping from variables to set of constraints, such that `var_to_const[var]` is the set of constraints with `var` in their scope.

cspProblem.py — (continued)

```
46 class CSP(object):
47     """A CSP consists of
48     * a title (a string)
49     * variables, a list or set of variables
50     * constraints, a list of constraints
51     * var_to_const, a variable to set of constraints dictionary
```

```
    """
    def __init__(self, title, variables, constraints):
        """title is a string
        variables is set of variables
        constraints is a list of constraints
        """
        self.title = title
        self.variables = variables
        self.constraints = constraints
        self.var_to_const = {var:set() for var in self.variables}
        for con in constraints:
            for var in con.scope:
                self.var_to_const[var].add(con)

    def __str__(self):
        """string representation of CSP"""
        return self.title

    def __repr__(self):
        """more detailed string representation of CSP"""
        return f"CSP({self.title}, {self.variables}, {([str(c) for c in
            self.constraints])})"
```

`csp.consistent(assignment)` returns true if the assignment is consistent with each of the constraints in `csp` (i.e., all of the constraints that can be evaluated evaluate to true). Unless the assignment assigns to all variables, `consistent` does *not* imply the CSP is consistent or has a solution, because constraints involving variables not in the assignment are ignored.

cspProblem.py — (continued)

```
    def consistent(self,assignment):
        """assignment is a variable:value dictionary
        returns True if all of the constraints that can be evaluated
                        evaluate to True given assignment.
        """
        return all(con.holds(assignment)
                   for con in self.constraints
                   if con.can_evaluate(assignment))
```

The **show** method uses matplotlib to show the graphical structure of a constraint network. This also includes code used for the consistency GUI (Section 4.4.2).

cspProblem.py — (continued)

```
    def show(self, linewidth=3, showDomains=False, showAutoAC = False):
        self.linewidth = linewidth
        self.picked = None
        plt.ion()  # interactive
        self.arcs = {} # arc: (con,var) dictionary
        self.thelines = {} # (con,var):arc dictionary
        self.nodes = {} # node: variable dictionary
```

```
        self.fig, self.ax= plt.subplots(1, 1)
        self.ax.set_axis_off()
        for var in self.variables:
            if var.position is None:
                var.position = (random.random(), random.random())
        self.showAutoAC = showAutoAC # used for consistency GUI
        self.autoAC = False
        domains = {var:var.domain for var in self.variables} if showDomains
            else {}
        self.draw_graph(domains=domains)

    def draw_graph(self, domains={}, to_do = {}, title=None, fontsize=10):
        self.ax.clear()
        self.ax.set_axis_off()
        if title:
            self.ax.set_title(title, fontsize=fontsize)
        else:
            self.ax.set_title(self.title, fontsize=fontsize)
        var_bbox = dict(boxstyle="round4,pad=1.0,rounding_size=0.5",
                        facecolor="yellow")
        con_bbox = dict(boxstyle="square,pad=1.0",facecolor="lightyellow")
        self.autoACtext = self.ax.text(0,0,"Auto AC" if self.showAutoAC
            else "",
                                       bbox={'boxstyle':'square,pad=1.0',
                                             'facecolor':'pink'},
                                       picker=True, fontsize=fontsize)
        for con in self.constraints:
            if con.position is None:
                con.position = tuple(sum(var.position[i] for var in
                    con.scope)/len(con.scope)
                                     for i in range(2))
            cx,cy = con.position
            bbox = con_bbox
            for var in con.scope:
                vx,vy = var.position
                if (var,con) in to_do:
                    color = 'blue'
                else:
                    color = 'green'
                line = lines.Line2D([cx,vx], [cy,vy], axes=self.ax,
                    color=color,
                                    picker=True, pickradius=10,
                                        linewidth=self.linewidth)
                self.arcs[line]= (var,con)
                self.thelines[(var,con)] = line
                self.ax.add_line(line)
            self.ax.text(cx,cy,con.string,
                                 bbox=con_bbox,
                                 ha='center',va='center', fontsize=fontsize)
        for var in self.variables:
```

```
            x,y = var.position
            if domains:
                node_label = f"{var.name}\n{domains[var]}"
            else:
                node_label = var.name
            node = self.ax.text(x, y, node_label, bbox=var_bbox,
                ha='center', va='center',
                        picker=True, fontsize=fontsize)
            self.nodes[node] = var
        self.fig.canvas.mpl_connect('pick_event', self.pick_handler)
```

The following method is used for the GUI (Section 4.4.2).

cspProblem.py — (continued)

```
    def pick_handler(self,event):
        mouseevent = event.mouseevent
        self.last_artist = artist = event.artist
        #print('***picker handler:',artist, 'mouseevent:', mouseevent)
        if artist in self.arcs:
            #print('### selected arc',self.arcs[artist])
            self.picked = self.arcs[artist]
        elif artist in self.nodes:
            #print('### selected node',self.nodes[artist])
            self.picked = self.nodes[artist]
        elif artist==self.autoACtext:
            self.autoAC = True
            #print("*** autoAC")
        else:
            print("### unknown click")
```

4.1.4 Examples

In the following code `ne\_`, when given a number, returns a function that is true when its argument is not that number. For example, if `f=ne\_(3)`, then `f(2)` is True and `f(3)` is False. That is, `ne\_`$(x)(y)$ is true when $x \neq y$. Allowing a function of multiple arguments to use its arguments one at a time is called **currying**, after the logician Haskell Curry. Some alternative implementations are commented out; the uncommented one allows the partial functions to have names.

cspExamples.py — Example CSPs

```
from cspProblem import Variable, CSP, Constraint
from operator import lt,ne,eq,gt

def ne_(val):
    """not equal value"""
    # return lambda x: x != val # alternative definition
    # return partial(ne,val)  # another alternative definition
    def nev(x):
```

```
        return val != x
    nev.__name__ = f"{val} != "   # name of the function
    return nev
```

Similarly $is\_(x)(y)$ is true when $x = y$.

cspExamples.py — (continued)

```
def is_(val):
    """is a value"""
    # return lambda x: x == val # alternative definition
    # return partial(eq,val)   # another alternative definition
    def isv(x):
        return val == x
    isv.__name__ = f"{val} == "
    return isv
```

`csp0` has variables X, Y and Z, each with domain $\{1, 2, 3\}$. The constraints are $X < Y$ and $Y < Z$.

cspExamples.py — (continued)

```
X = Variable('X', {1,2,3}, position=(0.1,0.8))
Y = Variable('Y', {1,2,3}, position=(0.5,0.2))
Z = Variable('Z', {1,2,3}, position=(0.9,0.8))
csp0 = CSP("csp0", {X,Y,Z},
          [ Constraint([X,Y], lt, "X<Y"),
            Constraint([Y,Z], lt, "Y<Z")])
```

`csp1` has variables A, B and C, each with domain $\{1, 2, 3, 4\}$. The constraints are $A < B$, $B \neq 2$, and $B < C$. This is slightly more interesting than `csp0` as it has more solutions. This example is used in the unit tests, and so if it is changed, the unit tests need to be changed. `csp1s` is the same, but with only the constraints $A < B$ and $B < C$

cspExamples.py — (continued)

```
A = Variable('A', {1,2,3,4}, position=(0.2,0.9))
B = Variable('B', {1,2,3,4}, position=(0.8,0.9))
C = Variable('C', {1,2,3,4}, position=(1,0.3))
C0 = Constraint([A,B], lt, "A < B", position=(0.4,0.3))
C1 = Constraint([B], ne_(2), "B != 2", position=(1,0.7))
C2 = Constraint([B,C], lt, "B < C", position=(0.6,0.1))
csp1 = CSP("csp1", {A, B, C},
          [C0, C1, C2])

csp1s = CSP("csp1s", {A, B, C},
          [C0, C2]) # A<B, B<C
```

The next CSP, *csp*2 is Example 4.9 of Poole and Mackworth [2023]; the domain consistent network (after applying the unary constraints) is shown in Figure 4.2. Note that we use the same variables as the previous example and add two more.

cspExamples.py — (continued)

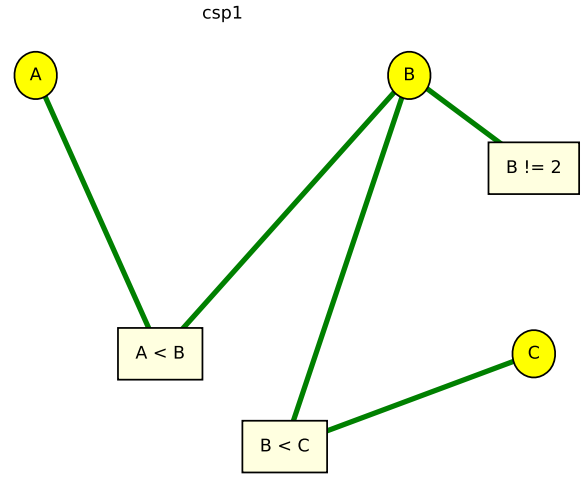


Figure 4.1: csp1.show()

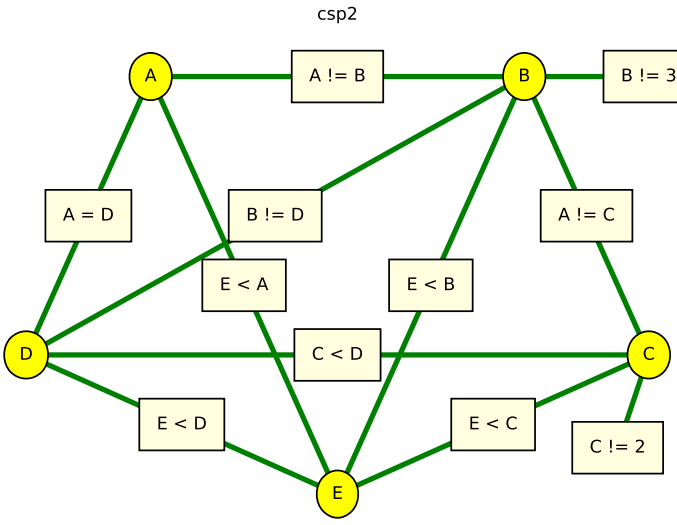


Figure 4.2: csp2.show()

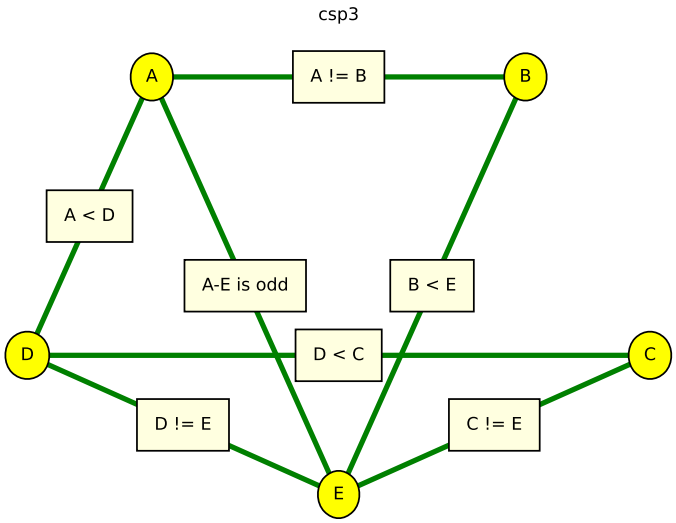


Figure 4.3: csp3.show()

```
D = Variable('D', {1,2,3,4}, position=(0,0.3))
E = Variable('E', {1,2,3,4}, position=(0.5,0))
csp2 = CSP("csp2", {A,B,C,D,E},
          [ Constraint([B], ne_(3), "B != 3", position=(1,0.9)),
           Constraint([C], ne_(2), "C != 2", position=(0.95,0.1)),
           Constraint([A,B], ne, "A != B"),
           Constraint([B,C], ne, "A != C"),
           Constraint([C,D], lt, "C < D"),
           Constraint([A,D], eq, "A = D"),
           Constraint([E,A], lt, "E < A"),
           Constraint([E,B], lt, "E < B"),
           Constraint([E,C], lt, "E < C"),
           Constraint([E,D], lt, "E < D"),
           Constraint([B,D], ne, "B != D")])
```

The following example is another scheduling problem (but with multiple answers). This is the same as "scheduling 2" in the original AIspace.org consistency app.

cspExamples.py — (continued)

```
csp3 = CSP("csp3", {A,B,C,D,E},
          [Constraint([A,B], ne, "A != B"),
           Constraint([A,D], lt, "A < D"),
           Constraint([A,E], lambda a,e: (a-e)%2 == 1, "A-E is odd"),
           Constraint([B,E], lt, "B < E"),
           Constraint([D,C], lt, "D < C"),
           Constraint([C,E], ne, "C != E"),
```

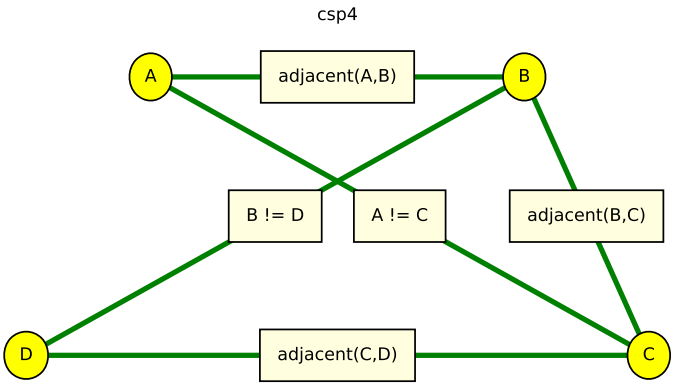


Figure 4.4: csp4.show()

```
73            Constraint([D,E], ne, "D != E")])
```

The following example is another abstract scheduling problem. What are the solutions?

cspExamples.py — (continued)

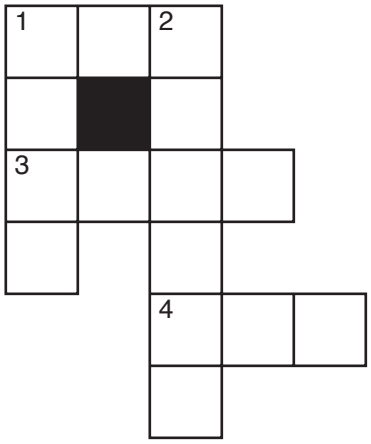
```
75 def adjacent(x,y):
76    """True when x and y are adjacent numbers"""
77    return abs(x-y) == 1
78
79 csp4 = CSP("csp4", {A,B,C,D},
80           [Constraint([A,B], adjacent, "adjacent(A,B)"),
81            Constraint([B,C], adjacent, "adjacent(B,C)"),
82            Constraint([C,D], adjacent, "adjacent(C,D)"),
83            Constraint([A,C], ne, "A != C"),
84            Constraint([B,D], ne, "B != D") ])
```

The following examples represent the crossword shown in Figure 4.5.

In the first representation, the variables represent words. The constraint imposed by the crossword is that where two words intersect, the letter at the intersection must be the same. The method `meet_at` is used to test whether two words intersect with the same letter. For example, the constraint `meet_at(2,0)` means that the third letter (at position 2) of the first argument is the same as the first letter of the second argument. This is shown in Figure 4.6.

cspExamples.py — (continued)

```
86 def meet_at(p1,p2):
```



Words:
ant, big, bus, car, has, book, buys, hold, lane, year, ginger, search, symbol, syntax.

Figure 4.5: crossword1: a crossword puzzle to be solved

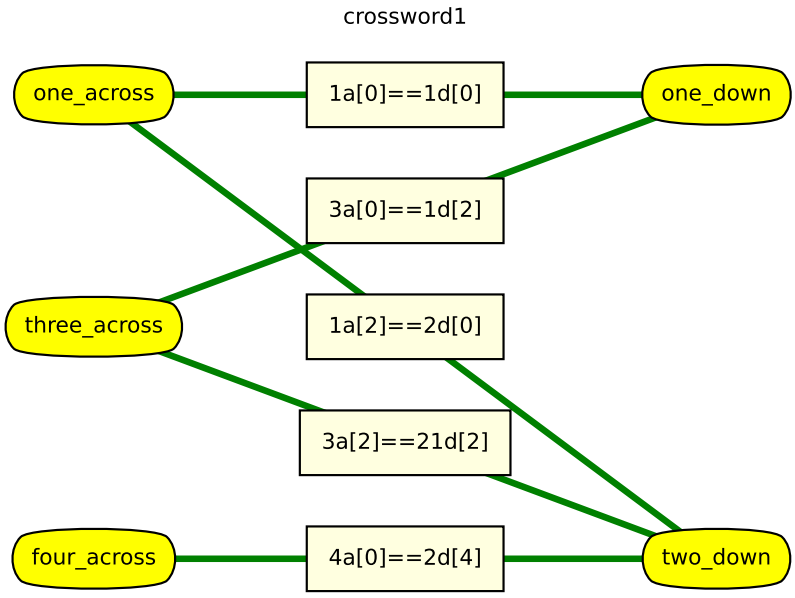


Figure 4.6: crossword1.show()

```
87      """returns a function of two words that is true
88                  when the words intersect at positions p1, p2.
89      The positions are relative to the words; starting at position 0.
90      meet_at(p1,p2)(w1,w2) is true if the same letter is at position p1 of
            word w1
91           and at position p2 of word w2.
92      """
93      def meets(w1,w2):
94          return w1[p1] == w2[p2]
95      meets.__name__ = f"meet_at({p1},{p2})"
96      return meets
97
98  one_across = Variable('one_across', {'ant', 'big', 'bus', 'car', 'has'},
        position=(0.1,0.9))
99  one_down = Variable('one_down', {'book', 'buys', 'hold', 'lane', 'year'},
        position=(0.9,0.9))
100 two_down = Variable('two_down', {'ginger', 'search', 'symbol', 'syntax'},
        position=(0.9,0.1))
101 three_across = Variable('three_across', {'book', 'buys', 'hold', 'land',
        'year'}, position=(0.1,0.5))
102 four_across = Variable('four_across',{'ant', 'big', 'bus', 'car', 'has'},
        position=(0.1,0.1))
103 crossword1 = CSP("crossword1",
104                 {one_across, one_down, two_down, three_across,
                        four_across},
105                 [Constraint([one_across,one_down],
                        meet_at(0,0),"1a[0]==1d[0]"),
106                  Constraint([one_across,two_down],
                        meet_at(2,0),"1a[2]==2d[0]"),
107                  Constraint([three_across,two_down],
                        meet_at(2,2),"3a[2]==21d[2]"),
108                  Constraint([three_across,one_down],
                        meet_at(0,2),"3a[0]==1d[2]"),
109                  Constraint([four_across,two_down],
                        meet_at(0,4),"4a[0]==2d[4]")
110                 ])
```

In an alternative representation of a crossword (the "dual" representation), the variables represent letters, and the constraints are that adjacent sequences of letters form words. This is shown in Figure 4.7.

cspExamples.py — (continued)

```
112 words = {'ant', 'big', 'bus', 'car', 'has','book', 'buys', 'hold',
113         'lane', 'year', 'ginger', 'search', 'symbol', 'syntax'}
114
115 def is_word(*letters, words=words):
116     """is true if the letters concatenated form a word in words"""
117     return "".join(letters) in words
118
119 letters = {"a", "b", "c", "d", "e", "f", "g", "h", "i", "j", "k", "l",
120   "m", "n", "o", "p", "q", "r", "s", "t", "u", "v", "w", "x", "y",
```

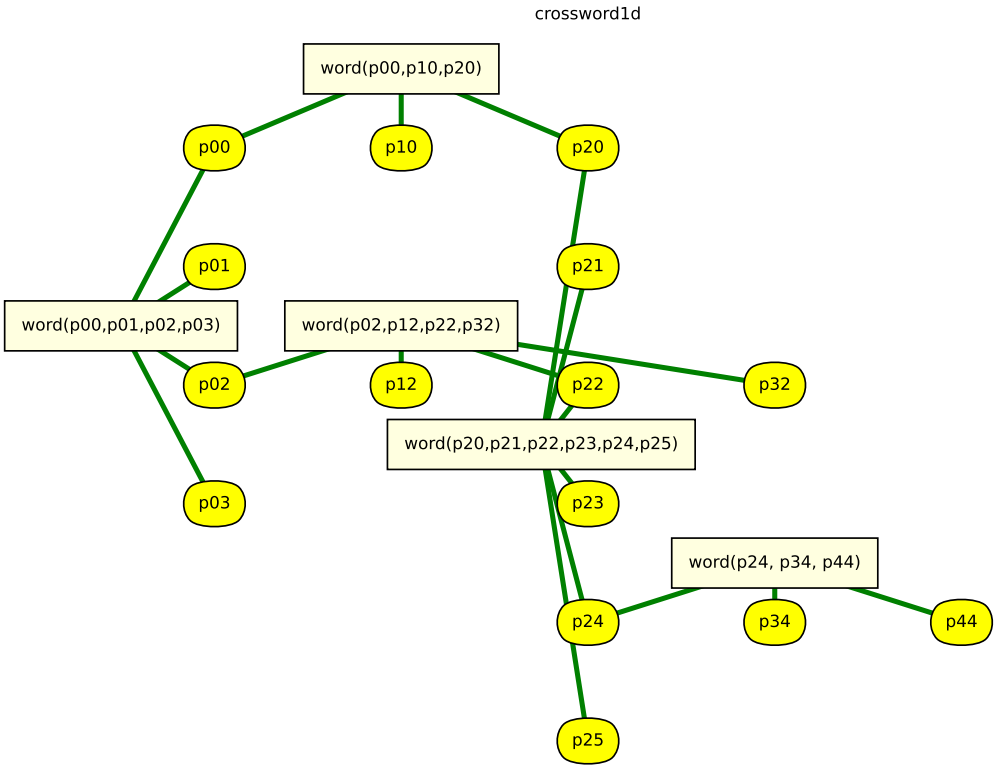


Figure 4.7: crossword1d.show()

```
121   "z"}
122
123 # pij is the variable representing the letter i from the left and j down
        (starting from 0)
124 p00 = Variable('p00', letters, position=(0.1,0.85))
125 p10 = Variable('p10', letters, position=(0.3,0.85))
126 p20 = Variable('p20', letters, position=(0.5,0.85))
127 p01 = Variable('p01', letters, position=(0.1,0.7))
128 p21 = Variable('p21', letters, position=(0.5,0.7))
129 p02 = Variable('p02', letters, position=(0.1,0.55))
130 p12 = Variable('p12', letters, position=(0.3,0.55))
131 p22 = Variable('p22', letters, position=(0.5,0.55))
132 p32 = Variable('p32', letters, position=(0.7,0.55))
133 p03 = Variable('p03', letters, position=(0.1,0.4))
134 p23 = Variable('p23', letters, position=(0.5,0.4))
135 p24 = Variable('p24', letters, position=(0.5,0.25))
136 p34 = Variable('p34', letters, position=(0.7,0.25))
137 p44 = Variable('p44', letters, position=(0.9,0.25))
138 p25 = Variable('p25', letters, position=(0.5,0.1))
139
140 crossword1d = CSP("crossword1d",
```

```
                {p00, p10, p20, # first row
                 p01, p21, # second row
                 p02, p12, p22, p32, # third row
                 p03, p23, #fourth row
                 p24, p34, p44, # fifth row
                 p25 # sixth row
                 },
                [Constraint([p00, p10, p20], is_word, "word(p00,p10,p20)",
                            position=(0.3,0.95)), #1-across
                 Constraint([p00, p01, p02, p03], is_word,
                     "word(p00,p01,p02,p03)",
                            position=(0,0.625)), # 1-down
                 Constraint([p02, p12, p22, p32], is_word,
                     "word(p02,p12,p22,p32)",
                            position=(0.3,0.625)), # 3-across
                 Constraint([p20, p21, p22, p23, p24, p25], is_word,
                     "word(p20,p21,p22,p23,p24,p25)",
                            position=(0.45,0.475)), # 2-down
                 Constraint([p24, p34, p44], is_word, "word(p24, p34,
                     p44)",
                            position=(0.7,0.325)) # 4-across
                 ])
```

Exercise 4.1 How many assignments of a value to each variable are there for each of the representations of the above crossword? Do you think an exhaustive enumeration will work for either one?

The queens problem is a puzzle on a chess board, where the idea is to place a queen on each column so the queens cannot take each other: there are no two queens on the same row, column or diagonal. The **n-queens problem** is a generalization where the size of the board is an $n \times n$, and n queens have to be placed.

Here is a representation of the n-queens problem, where the variables are the columns and the values are the rows in which the queen is placed. The original queens problem on a standard (8×8) chess board is `n_queens(8)`

cspExamples.py — (continued)

```
def queens(ri,rj):
    """ri and rj are different rows, return the condition that the queens
        cannot take each other"""
    def no_take(ci,cj):
        """is true if queen at (ri,ci) cannot take a queen at (rj,cj)"""
        return ci != cj and abs(ri-ci) != abs(rj-cj)
    return no_take

def n_queens(n):
    """returns a CSP for n-queens"""
    columns = list(range(n))
    variables = [Variable(f"R{i}",columns) for i in range(n)]
      # note positions will be random
```

```
172     return CSP("n-queens",
173                variables,
174                [Constraint([variables[i], variables[j]], queens(i,j),"" )
175                     for i in range(n) for j in range(n) if i != j])
176
177 # try the CSP n_queens(8) in one of the solvers.
178 # What is the smallest n for which there is a solution?
```

Exercise 4.2 How many constraints does this representation of the n-queens problem produce? Can it be done with fewer constraints? Either explain why it can't be done with fewer constraints, or give a solution using fewer constraints.

Unit tests

The following defines a **unit test** for csp solvers, by default using example csp1.

cspExamples.py — (continued)

```
180 def test_csp(CSP_solver, csp=csp1,
181             solutions=[{A: 1, B: 3, C: 4}, {A: 2, B: 3, C: 4}]):
182     """CSP_solver is a solver that takes a csp and returns a solution
183     csp is a constraint satisfaction problem
184     solutions is the list of all solutions to csp
185     This tests whether the solution returned by CSP_solver is a solution.
186     """
187     print("Testing csp with",CSP_solver.__doc__)
188     sol0 = CSP_solver(csp)
189     print("Solution found:",sol0)
190     assert sol0 in solutions, f"Solution not correct for {csp}"
191     print("Passed unit test")
```

Exercise 4.3 Modify *test* so that instead of taking in a list of solutions, it checks whether the returned solution actually is a solution.

Exercise 4.4 Propose a test that is appropriate for CSPs with no solutions. Assume that the test designer knows there are no solutions. Consider what a CSP solver should return if there are no solutions to the CSP.

Exercise 4.5 Write a unit test that checks whether all solutions (e.g., for the search algorithms that can return multiple solutions) are correct, and whether all solutions can be found.

4.2 A Simple Depth-first Solver

The first solver carries out a depth-first search through the space of partial assignments. This takes in a CSP problem and an optional variable ordering (a list of the variables in the CSP). It returns a generator of the solutions (see Section 1.5.3 on `yield` for enumerations).

cspDFS.py — Solving a CSP using depth-first search.

```
11 import cspExamples
```

```
12
13 def dfs_solver(constraints, context, var_order):
14     """generator for all solutions to csp.
15     context is an assignment of values to some of the variables.
16     var_order is  a list of the variables in csp that are not in context.
17     """
18     to_eval = {c for c in constraints if c.can_evaluate(context)}
19     if all(c.holds(context) for c in to_eval):
20         if var_order == []:
21             yield context
22         else:
23             rem_cons = [c for c in constraints if c not in to_eval]
24             var = var_order[0]
25             for val in var.domain:
26                 yield from dfs_solver(rem_cons, context|{var:val},
                       var_order[1:])
27
28 def dfs_solve_all(csp, var_order=None):
29     """depth-first CSP solver to return a list of all solutions to csp.
30     """
31     if var_order == None:  # use an arbitrary variable order
32         var_order = list(csp.variables)
33     return list( dfs_solver(csp.constraints, {}, var_order))
34
35 def dfs_solve1(csp, var_order=None):
36     """depth-first CSP solver"""
37     if var_order == None:  # use an arbitrary variable order
38         var_order = list(csp.variables)
39     for sol in dfs_solver(csp.constraints, {}, var_order):
40         return sol #return first one
41
42 if __name__ == "__main__":
43     cspExamples.test_csp(dfs_solve1)
44
45 #Try:
46 # dfs_solve_all(cspExamples.csp1)
47 # dfs_solve_all(cspExamples.csp2)
48 # dfs_solve_all(cspExamples.crossword1)
49 # dfs_solve_all(cspExamples.crossword1d) # warning: may take a *very* long
      time!
```

Exercise 4.6 Instead of testing all constraints at every node, change it so each constraint is only tested when all of its variables are assigned. Given an elimination ordering, it is possible to determine when each constraint needs to be tested. Implement this. Hint: create a parallel list of sets of constraints, where at each position i in the list, the constraints at position i can be evaluated when the variable at position i has been assigned.

Exercise 4.7 Estimate how long `dfs_solve_all(crossword1d)` will take on your computer. To do this, reduce the number of variables that need to be assigned, so that the simplified problem can be solved in a reasonable time (between 0.1

second and 10 seconds). This can be done by reducing the number of variables in `var_order`, as the program only splits on these. How much more time will it take if the number of variables is increased by 1? (Try it!) Then extrapolate to all of the variables. See Section 1.6.1 for how to time your code. Would making the code 100 times faster or using a computer 100 times faster help?

4.3 Converting CSPs to Search Problems

To run the demo, in folder "aipython", load "cspSearch.py", and copy and paste the example queries at the bottom of that file.

The next solver constructs a search space that can be solved using the search methods of the previous chapter. This takes in a CSP problem and an optional variable ordering, which is a list of the variables in the CSP. In this search space:

- A node is a *variable* : *value* dictionary which does not violate any constraints (so that dictionaries that violate any conmtratints are not added).
- An arc corresponds to an assignment of a value to the next variable. This assumes a static ordering; the next variable chosen to split does not depend on the context. If no variable ordering is given, this makes no attempt to choose a good ordering.

cspSearch.py — Representations of a Search Problem from a CSP.

```
11 from cspProblem import CSP, Constraint
12 from searchProblem import Arc, Search_problem
13
14 class Search_from_CSP(Search_problem):
15     """A search problem directly from the CSP.
16
17     A node is a variable:value dictionary"""
18     def __init__(self, csp, variable_order=None):
19         self.csp=csp
20         if variable_order:
21             assert set(variable_order) == set(csp.variables)
22             assert len(variable_order) == len(csp.variables)
23             self.variables = variable_order
24         else:
25             self.variables = list(csp.variables)
26
27     def is_goal(self, node):
28         """returns whether the current node is a goal for the search
29         """
30         return len(node)==len(self.csp.variables)
31
32     def start_node(self):
33         """returns the start node for the search
```

```
        """
        return {}
```

The *neighbors*(*node*) method uses the fact that the length of the node, which is the number of variables already assigned, is the index of the next variable to split on. Note that we do not need to check whether there are no more variables to split on, as the nodes are all consistent, by construction, and so when there are no more variables we have a solution, and so don't need the neighbors.

cspSearch.py — (continued)

```
    def neighbors(self, node):
        """returns a list of the neighboring nodes of node.
        """
        var = self.variables[len(node)] # the next variable
        res = []
        for val in var.domain:
            new_env = node|{var:val} #dictionary union
            if self.csp.consistent(new_env):
                res.append(Arc(node,new_env))
        return res
```

The unit tests relies on a solver. The following procedure creates a solver using search that can be tested.

cspSearch.py — (continued)

```
import cspExamples
from searchGeneric import Searcher

def solver_from_searcher(csp):
    """depth-first search solver"""
    path = Searcher(Search_from_CSP(csp)).search()
    if path is not None:
        return path.end()
    else:
        return None

if __name__ == "__main__":
    test_csp(solver_from_searcher)

## Test Solving CSPs with Search:
searcher1 = Searcher(Search_from_CSP(cspExamples.csp1))
#print(searcher1.search()) # get next solution
searcher2 = Searcher(Search_from_CSP(cspExamples.csp2))
#print(searcher2.search()) # get next solution
searcher3 = Searcher(Search_from_CSP(cspExamples.crossword1))
#print(searcher3.search()) # get next solution
searcher4 = Searcher(Search_from_CSP(cspExamples.crossword1d))
#print(searcher4.search()) # get next solution (warning: slow)
```

Exercise 4.8 What would happen if we constructed the new assignment by assigning $node[var] = val$ (with side effects) instead of using dictionary union? Give

an example of where this could give a wrong answer. How could the algorithm be changed to work with side effects? (Hint: think about what information needs to be in a node).

Exercise 4.9 Change neighbors so that it returns an iterator of values rather than a list. (Hint: use *yield*.)

4.4 Consistency Algorithms

To run the demo, in folder "aipython", load "cspConsistency.py", and copy and paste the commented-out example queries at the bottom of that file.

A *Con_solver* is used to simplify a CSP using arc consistency.

cspConsistency.py — Arc Consistency and Domain splitting for solving a CSP

```
from display import Displayable

class Con_solver(Displayable):
    """Solves a CSP with arc consistency and domain splitting
    """
    def __init__(self, csp):
        """a CSP solver that uses arc consistency
        * csp is the CSP to be solved
        """
        self.csp = csp
```

The following implementation of arc consistency maintains the set *to_do* of (variable, constraint) pairs that are to be checked. It takes in a domain dictionary and returns a new domain dictionary. It needs to be careful to avoid side effects; this is implemented here by copying the *domains* dictionary and the *to_do* set.

cspConsistency.py — (continued)

```
    def make_arc_consistent(self, domains=None, to_do=None):
        """Makes this CSP arc-consistent using generalized arc consistency
        domains is a variable:domain dictionary
        to_do is a set of (variable,constraint) pairs
        returns the reduced domains (an arc-consistent variable:domain
            dictionary)
        """
        if domains is None:
            self.domains = {var:var.domain for var in self.csp.variables}
        else:
            self.domains = domains.copy() # use a copy of domains
        if to_do is None:
            to_do = {(var, const) for const in self.csp.constraints
                    for var in const.scope}
        else:
```

```
36              to_do = to_do.copy() # use a copy of to_do
37          self.display(5,"Performing AC with domains", self.domains)
38          while to_do:
39              self.arc_selected = (var, const) = self.select_arc(to_do)
40              self.display(5, "Processing arc (", var, ",", const, ")")
41              other_vars = [ov for ov in const.scope if ov != var]
42              new_domain = {val for val in self.domains[var]
43                            if self.any_holds(self.domains, const, {var:
                                  val}, other_vars)}
44              if new_domain != self.domains[var]:
45                  self.add_to_do = self.new_to_do(var, const) - to_do
46                  self.display(3, f"Arc: ({var}, {const}) is inconsistent\n"
47                               f"Domain pruned, dom({var}) ={new_domain} due to
                                     {const}")
48                  self.domains[var] = new_domain
49                  self.display(4, " adding", self.add_to_do if self.add_to_do
50                                 else "nothing", "to to_do.")
51                  to_do |= self.add_to_do   # set union
52              self.display(5, f"Arc: ({var},{const}) now consistent")
53          self.display(5, "AC done. Reduced domains", self.domains)
54          return self.domains
55
56      def new_to_do(self, var, const):
57          """returns new elements to be added to to_do after assigning
58          variable var in constraint const.
59          """
60          return {(nvar, nconst) for nconst in self.csp.var_to_const[var]
61                  if nconst != const
62                  for nvar in nconst.scope
63                  if nvar != var}
```

The following selects an arc. Any element of *to_do* can be selected. The selected element needs to be removed from *to_do*. The default implementation just selects which ever element *pop* method for sets returns. The graphical user interface below allows the user to select an arc. Alternatively, a more sophisticated selection could be employed.

cspConsistency.py — (continued)

```
65      def select_arc(self, to_do):
66          """Selects the arc to be taken from to_do .
67          * to_do is a set of arcs, where an arc is a (variable,constraint)
                pair
68          the element selected must be removed from to_do.
69          """
70          return to_do.pop()
```

The value of `new_domain` is the subset of the domain of `var` that is consistent with the assignment to the other variables. To make it easier to understand, the following treats unary (with no other variables in the constraint) and binary (with one other variables in the constraint) constraints as special cases. These cases are not strictly necessary; the last case covers the first two cases, but is

more difficult to understand without seeing the first two cases. Note that this case analysis is not in the code distribution, but can replace the assignment to `new_domain` above.

```
            if len(other_vars)==0:          # unary constraint
                new_domain = {val for val in self.domains[var]
                              if const.holds({var:val})}
            elif len(other_vars)==1:        # binary constraint
                other = other_vars[0]
                new_domain = {val for val in self.domains[var]
                       if any(const.holds({var: val,other:other_val})
                              for other_val in self.domains[other])}
            else:                           # general case
                new_domain = {val for val in self.domains[var]
                    if self.any_holds(self.domains, const, {var: val}, other_vars)}
```

any_holds is a recursive function that tries to finds an assignment of values to the other variables (*other_vars*) that satisfies constraint *const* given the assignment in *env*. The integer variable *ind* specifies which index to *other_vars* needs to be checked next. As soon as one assignment returns *True,* the algorithm returns *True*.

cspConsistency.py — (continued)

```
72      def any_holds(self, domains, const, env, other_vars, ind=0):
73          """returns True if Constraint const holds for an assignment
74          that extends env with the variables in other_vars[ind:]
75          env is a dictionary
76          """
77          if ind == len(other_vars):
78              return const.holds(env)
79          else:
80              var = other_vars[ind]
81              for val in domains[var]:
82                  if self.any_holds(domains, const, env|{var:val}, other_vars,
                        ind + 1):
83                      return True
84              return False
```

4.4.1 Direct Implementation of Domain Splitting

The following is a direct implementation of domain splitting with arc consistency. It implements the generator interface of Python (see Section 1.5.3). When it has found a solution it `yield`s the result; otherwise it recursively splits a domain (using `yield from`).

cspConsistency.py — (continued)

```
86      def generate_sols(self, domains=None, to_do=None, context=dict()):
87          """return list of all solution to the current CSP
```

```
        to_do is the list of arcs to check
        context is a dictionary of splits made (used for display)
        """
        new_domains = self.make_arc_consistent(domains, to_do)
        if any(len(new_domains[var]) == 0 for var in new_domains):
            self.display(1,f"No solutions for context {context}")
        elif all(len(new_domains[var]) == 1 for var in new_domains):
            self.display(1, "solution:", str({var: select(
                new_domains[var]) for var in new_domains}))
            yield {var: select(new_domains[var]) for var in new_domains}
        else:
            var = self.select_var(x for x in self.csp.variables if
                len(new_domains[x]) > 1)
            dom1, dom2 = partition_domain(new_domains[var])
            self.display(5, "...splitting", var, "into", dom1, "and", dom2)
            new_doms1 = new_domains | {var:dom1}
            new_doms2 = new_domains | {var:dom2}
            to_do = self.new_to_do(var, None)
            self.display(4, " adding", to_do if to_do else "nothing", "to
                to_do.")
            yield from self.generate_sols(new_doms1, to_do,
                context|{var:dom1})
            yield from self.generate_sols(new_doms2, to_do,
                context|{var:dom1})

    def solve_all(self, domains=None, to_do=None):
        return list(self.generate_sols())

    def solve_one(self, domains=None, to_do=None):
        return select(self.generate_sols())

    def select_var(self, iter_vars):
        """return the next variable to split"""
        return select(iter_vars)

def partition_domain(dom):
    """partitions domain dom into two.
    """
    split = len(dom) // 2
    dom1 = set(list(dom)[:split])
    dom2 = dom - dom1
    return dom1, dom2
```

cspConsistency.py — (continued)

```
def select(iterable):
    """select an element of iterable.
    Returns None if there is no such element.

    This implementation just picks the first element.
    For many uses, which element is selected does not affect correctness,
```

```
133     but may affect efficiency.
134     """
135     for e in iterable:
136         return e # returns first element found
```

Exercise 4.10 Implement *solve_all* that returns the set of all solutions without using `yield`. Hint: it can be like `generate_sols` but returns a set of solutions; the recursive calls can be unioned; `|` is Python's union.

Exercise 4.11 Implement *solve_one* that returns one solution if one exists, or `False` otherwise, without using `yield`. Hint: Python's "or" has the behavior `A or B` will return the value of `A` unless it is None or False, in which case the value of `B` is returned.

Unit test:

cspConsistency.py — (continued)

```
138 import cspExamples
139 def ac_solver(csp):
140     "arc consistency (ac_solver)"
141     for sol in Con_solver(csp).generate_sols():
142         return sol
143 
144 if __name__ == "__main__":
145     cspExamples.test_csp(ac_solver)
```

4.4.2 Consistency GUI

The consistency GUI allows students to step through the algorithm, choosing which arc to process next, and which variable to split.

Figure 4.8 shows the state of the GUI after two arcs have been made arc consistent. The arcs on the `to_do` list arc colored blue. The green arcs are those have been made arc consistent. The user can click on a blue arc to process that arc. If the arc selected is not arc consistent, it is made red, the domain is reduced, and then the arc becomes green. If the arc was already arc consistent it turns green.

This is implemented by overriding `select_arc` and `select_var` to allow the user to pick the arcs and the variables, and overriding `display` to allow for the animation. Note that the first argument of `display` (the number) in the code above is interpreted with a special meaning by the GUI and should only be changed with care.

Clicking `AutoAC` automates arc selection until the network is arc consistent.

cspConsistencyGUI.py — GUI for consistency-based CSP solving

```
11 from cspConsistency import Con_solver
12 import matplotlib.pyplot as plt
13 
14 class ConsistencyGUI(Con_solver):
15     def __init__(self, csp, fontsize=10, speed=1, **kwargs):
```

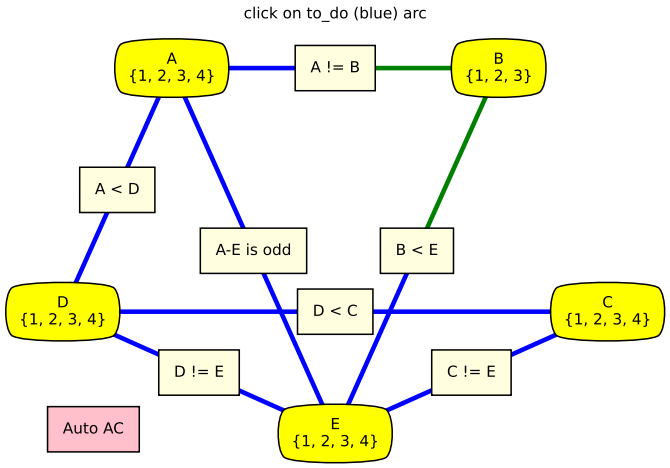


Figure 4.8: ConsistencyGUI(cspExamples.csp3).go()

```
        """
        csp is the csp to show
        fontsize is the size of the text
        speed is the number of animations per second (controls delay_time)
            1 (slow) and 4 (fast) seem like good values
        """
        self.fontsize = fontsize
        self.delay_time = 1/speed
        self.quitting = False
        Con_solver.__init__(self, csp, **kwargs)
        csp.show(showAutoAC = True)
        csp.fig.canvas.mpl_connect('close_event', self.window_closed)

    def go(self):
        try:
            res = self.solve_all()
            self.csp.draw_graph(domains=self.domains,
                                title="No more solutions. GUI finished. ",
                                fontsize=self.fontsize)
            return res
        except ExitToPython:
            print("GUI closed")

    def select_arc(self, to_do):
        while True:
            self.csp.draw_graph(domains=self.domains, to_do=to_do,
                                title="click on to_do (blue) arc",
                                    fontsize=self.fontsize)
            self.wait_for_user()
```

```
            if self.csp.autoAC:
                break
            picked = self.csp.picked
            self.csp.picked = None
            if picked in to_do:
                to_do.remove(picked)
                print(f"{picked} picked")
                return picked
            else:
                print(f"{picked} not in to_do. Pick one of {to_do}")
        if self.csp.autoAC:
            self.csp.draw_graph(domains=self.domains, to_do=to_do,
                                title="Auto AC", fontsize=self.fontsize)
            plt.pause(self.delay_time)
            return to_do.pop()

    def select_var(self, iter_vars):
        vars = list(iter_vars)
        while True:
            self.csp.draw_graph(domains=self.domains,
                                title="Arc consistent. Click node to
                                    split",
                                fontsize=self.fontsize)
            self.csp.autoAC = False
            self.wait_for_user()
            picked = self.csp.picked
            self.csp.picked = None
            if picked in vars:
                #print("splitting",picked)
                return picked
            else:
                print(picked,"not in",vars)

    def display(self,n,*args,**nargs):
        if n <= self.max_display_level: # default display
            print(*args, **nargs)
        if n==1: # solution found or no solutions"
            self.csp.draw_graph(domains=self.domains, to_do=set(),
                                title=' '.join(args)+": click any node or
                                    arc to continue",
                                fontsize=self.fontsize)
            self.csp.autoAC = False
            self.wait_for_user()
            self.csp.picked = None
        elif n==2: # backtracking
            plt.title("backtracking: click any node or arc to continue")
            self.csp.autoAC = False
            self.wait_for_user()
            self.csp.picked = None
        elif n==3: # inconsistent arc
```

```
92              line = self.csp.thelines[self.arc_selected]
93              line.set_color('red')
94              line.set_linewidth(10)
95              plt.pause(self.delay_time)
96              line.set_color('limegreen')
97              line.set_linewidth(self.csp.linewidth)
98          #elif n==4 and self.add_to_do: # adding to to_do
99          #    print("adding to to_do",self.add_to_do) ## highlight these arc
100
101     def wait_for_user(self):
102         while self.csp.picked == None and not self.csp.autoAC and not
                self.quitting:
103             plt.pause(0.01) # controls reaction time of GUI
104         if self.quitting:
105             raise ExitToPython()
106
107     def window_closed(self, event):
108         self.quitting = True
109
110 class ExitToPython(Exception):
111     pass
112
113 import cspExamples
114 # Try:
115 # ConsistencyGUI(cspExamples.csp1).go()
116 # ConsistencyGUI(cspExamples.csp3).go()
117 # ConsistencyGUI(cspExamples.csp3, speed=4, fontsize=15).go()
118
119 if __name__ == "__main__":
120     print("Try e.g.: ConsistencyGUI(cspExamples.csp3).go()")
```

4.4.3 Domain Splitting as an interface to graph searching

An alternative implementation is to implement domain splitting in terms of the search abstraction of Chapter 3.

A node is a dictionary that maps the variables to their (pruned) domains..

cspConsistency.py — (continued)

```
147 from searchProblem import Arc, Search_problem
148
149 class Search_with_AC_from_CSP(Search_problem,Displayable):
150     """A search problem with arc consistency and domain splitting
151
152     A node is a CSP """
153     def __init__(self, csp):
154         self.cons = Con_solver(csp) #copy of the CSP
155         self.domains = self.cons.make_arc_consistent()
156
157     def is_goal(self, node):
```

```
158         """node is a goal if all domains have 1 element"""
159         return all(len(node[var])==1 for var in node)
160 
161     def start_node(self):
162         return self.domains
163 
164     def neighbors(self,node):
165         """returns the neighboring nodes of node.
166         """
167         neighs = []
168         var = select(x for x in node if len(node[x])>1)
169         if var:
170             dom1, dom2 = partition_domain(node[var])
171             self.display(2,"Splitting", var, "into", dom1, "and", dom2)
172             to_do = self.cons.new_to_do(var,None)
173             for dom in [dom1,dom2]:
174                 newdoms = node | {var:dom}
175                 cons_doms = self.cons.make_arc_consistent(newdoms,to_do)
176                 if all(len(cons_doms[v])>0 for v in cons_doms):
177                     # all domains are non-empty
178                     neighs.append(Arc(node,cons_doms))
179                 else:
180                     self.display(2,"...",var,"in",dom,"has no solution")
181         return neighs
```

Exercise 4.12 When splitting a domain, this code splits the domain into half, approximately in half (without any effort to make a sensible choice). Does it work better to split one element from a domain?

Unit test:

cspConsistency.py — (continued)

```
183 import cspExamples
184 from searchGeneric import Searcher
185 
186 def ac_search_solver(csp):
187     """arc consistency (search interface)"""
188     sol = Searcher(Search_with_AC_from_CSP(csp)).search()
189     if sol:
190         return {v:select(d) for (v,d) in sol.end().items()}
191 
192 if __name__ == "__main__":
193     cspExamples.test_csp(ac_search_solver)
```

Testing:

cspConsistency.py — (continued)

```
195 ## Test Solving CSPs with Arc consistency and domain splitting:
196 #Con_solver.max_display_level = 4 # display details of AC (0 turns off)
197 #Con_solver(cspExamples.csp1).solve_all()
198 #searcher1d = Searcher(Search_with_AC_from_CSP(cspExamples.csp1))
```

```
#print(searcher1d.search())
#Searcher.max_display_level = 2 # display search trace (0 turns off)
#searcher2c = Searcher(Search_with_AC_from_CSP(cspExamples.csp2))
#print(searcher2c.search())
#searcher3c = Searcher(Search_with_AC_from_CSP(cspExamples.crossword1))
#print(searcher3c.search())
#searcher4c = Searcher(Search_with_AC_from_CSP(cspExamples.crossword1d))
#print(searcher4c.search())
```

4.5 Solving CSPs using Stochastic Local Search

To run the demo, in folder "aipython", load "cspSLS.py", and copy and paste the commented-out example queries at the bottom of that file. This assumes Python 3. Some of the queries require matplotlib.

The following code implements the two-stage choice (select one of the variables that are involved in the most constraints that are violated, then a value), the any-conflict algorithm (select a variable that participates in a violated constraint) and a random choice of variable, as well as a probabilistic mix of the three.

Given a CSP, the stochastic local searcher (*SLSearcher*) creates the data structures:

- *variables_to_select* is the set of all of the variables with domain-size greater than one. For a variable not in this set, we cannot pick another value from that variable.
- *var_to_constraints* maps from a variable into the set of constraints it is involved in. Note that the inverse mapping from constraints into variables is part of the definition of a constraint.

cspSLS.py — Stochastic Local Search for Solving CSPs

```
from cspProblem import CSP, Constraint
from searchProblem import Arc, Search_problem
from display import Displayable
import random
import heapq

class SLSearcher(Displayable):
    """A search problem directly from the CSP..

    A node is a variable:value dictionary"""
    def __init__(self, csp):
        self.csp = csp
        self.variables_to_select = {var for var in self.csp.variables
                                    if len(var.domain) > 1}
```

```
25        # Create assignment and conflicts set
26        self.current_assignment = None # this will trigger a random restart
27        self.number_of_steps = 0 #number of steps after the initialization
```

restart creates a new total assignment, and constructs the set of conflicts (the constraints that are false in this assignment).

cspSLS.py — (continued)

```
29    def restart(self):
30        """creates a new total assignment and the conflict set
31        """
32        self.current_assignment = {var:random_choice(var.domain) for
33                                   var in self.csp.variables}
34        self.display(2,"Initial assignment",self.current_assignment)
35        self.conflicts = set()
36        for con in self.csp.constraints:
37            if not con.holds(self.current_assignment):
38                self.conflicts.add(con)
39        self.display(2,"Number of conflicts",len(self.conflicts))
40        self.variable_pq = None
```

The *search* method is the top-level searching algorithm. It can either be used to start the search or to continue searching. If there is no current assignment, it must create one. Note that, when counting steps, a restart is counted as one step, which is not appropriate for CSPs with many variables, as it is a relatively expensive operation for these cases.

This method selects one of two implementations. The argument *prob_best* is the probability of selecting a best variable (one involving the most conflicts). When the value of *prob_best* is positive, the algorithm needs to maintain a priority queue of variables and the number of conflicts (using *search_with_var_pq*). If the probability of selecting a best variable is zero, it does not need to maintain this priority queue (as implemented in *search_with_any_conflict*).

The argument *prob_anycon* is the probability that the any-conflict strategy is used (which selects a variable at random that is in a conflict), assuming that it is not picking a best variable. Note that for the probability parameters, any value less that zero acts like probability zero and any value greater than 1 acts like probability 1. This means that when $prob\_anycon = 1.0$, a best variable is chosen with probability *prob_best*, otherwise a variable in any conflict is chosen. A variable is chosen at random with probability $1 - prob\_anycon - prob\_best$ as long as that is positive.

This returns the number of steps needed to find a solution, or *None* if no solution is found. If there is a solution, it is in *self.current_assignment*.

cspSLS.py — (continued)

```
42    def search(self,max_steps, prob_best=0, prob_anycon=1.0):
43        """
44        returns the number of steps or None if these is no solution.
45        If there is a solution, it can be found in self.current_assignment
46
```

```
47          max_steps is the maximum number of steps it will try before giving
                up
48          prob_best is the probability that a best variable (one in most
                conflict) is selected
49          prob_anycon is the probability that a variable in any conflict is
                selected
50          (otherwise a variable is chosen at random)
51          """
52          if self.current_assignment is None:
53              self.restart()
54              self.number_of_steps += 1
55              if not self.conflicts:
56                  self.display(1,"Solution found:", self.current_assignment,
                        "after restart")
57                  return self.number_of_steps
58          if prob_best > 0: # we need to maintain a variable priority queue
59              return self.search_with_var_pq(max_steps, prob_best,
                    prob_anycon)
60          else:
61              return self.search_with_any_conflict(max_steps, prob_anycon)
```

Exercise 4.13 This does an initial random assignment but does not do any random restarts. Implement a searcher that takes in the maximum number of walk steps (corresponding to existing *max_steps*) and the maximum number of restarts, and returns the total number of steps for the first solution found. (As in *search*, the solution found can be extracted from the variable *self.current_assignment*).

4.5.1 Any-conflict

In the any-conflict heuristic a variable that participates in a violated constraint is picked at random. The implementation need to keeps track of which variables are in conflicts. This is can avoid the need for a priority queue that is needed when the probability of picking a best variable is greater than zero.

cspSLS.py — (continued)

```
63      def search_with_any_conflict(self, max_steps, prob_anycon=1.0):
64          """Searches with the any_conflict heuristic.
65          This relies on just maintaining the set of conflicts;
66          it does not maintain a priority queue
67          """
68          self.variable_pq = None # we are not maintaining the priority queue.
69                                  # This ensures it is regenerated if
70                                  #   we call search_with_var_pq.
71          for i in range(max_steps):
72              self.number_of_steps +=1
73              if  random.random() < prob_anycon:
74                  con = random_choice(self.conflicts) # pick random conflict
75                  var = random_choice(con.scope) # pick variable in conflict
76              else:
77                  var = random_choice(self.variables_to_select)
```

```
78            if len(var.domain) > 1:
79                val = random_choice([val for val in var.domain
80                                     if val is not
                                         self.current_assignment[var]])
81                self.display(2,self.number_of_steps,":
                      Assigning",var,"=",val)
82                self.current_assignment[var]=val
83                for varcon in self.csp.var_to_const[var]:
84                    if varcon.holds(self.current_assignment):
85                        if varcon in self.conflicts:
86                            self.conflicts.remove(varcon)
87                    else:
88                        if varcon not in self.conflicts:
89                            self.conflicts.add(varcon)
90                self.display(2,"   Number of conflicts",len(self.conflicts))
91            if not self.conflicts:
92                self.display(1,"Solution found:", self.current_assignment,
93                               "in", self.number_of_steps,"steps")
94                return self.number_of_steps
95        self.display(1,"No solution in",self.number_of_steps,"steps",
96                    len(self.conflicts),"conflicts remain")
97        return None
```

Exercise 4.14 This makes no attempt to find the best value for the variable selected. Modify the code to include an option selects a value for the selected variable that reduces the number of conflicts the most. Have a parameter that specifies the probability that the best value is chosen, and otherwise chooses a value at random.

4.5.2 Two-Stage Choice

This is the top-level searching algorithm that maintains a priority queue of variables ordered by the number of conflicts, so that the variable with the most conflicts is selected first. If there is no current priority queue of variables, one is created.

The main complexity here is to maintain the priority queue. When a variable `var` is assigned a value `val`, for each constraint that has become satisfied or unsatisfied, each variable involved in the constraint need to have its count updated. The change is recorded in the dictionary *var_differential*, which is used to update the priority queue (see Section 4.5.3).

cspSLS.py — (continued)

```
99    def search_with_var_pq(self,max_steps, prob_best=1.0, prob_anycon=1.0):
100       """search with a priority queue of variables.
101       This is used to select a variable with the most conflicts.
102       """
103       if not self.variable_pq:
104           self.create_pq()
105       pick_best_or_con = prob_best + prob_anycon
```

```
        for i in range(max_steps):
            self.number_of_steps +=1
            randnum = random.random()
            ## Pick a variable
            if randnum < prob_best: # pick best variable
                var,oldval = self.variable_pq.top()
            elif randnum < pick_best_or_con: # pick a variable in a conflict
                con = random_choice(self.conflicts)
                var = random_choice(con.scope)
            else: #pick any variable that can be selected
                var = random_choice(self.variables_to_select)
            if len(var.domain) > 1: # var has other values
                ## Pick a value
                val = random_choice([val for val in var.domain if val is not
                                     self.current_assignment[var]])
                self.display(2,"Assigning",var,val)
                ## Update the priority queue
                var_differential = {}
                self.current_assignment[var]=val
                for varcon in self.csp.var_to_const[var]:
                    self.display(3,"Checking",varcon)
                    if varcon.holds(self.current_assignment):
                        if varcon in self.conflicts: # became consistent
                            self.display(3,"Became consistent",varcon)
                            self.conflicts.remove(varcon)
                            for v in varcon.scope: # v is in one fewer
                                conflicts
                                var_differential[v] =
                                    var_differential.get(v,0)-1
                    else:
                        if varcon not in self.conflicts: # was consis, not now
                            self.display(3,"Became inconsistent",varcon)
                            self.conflicts.add(varcon)
                            for v in varcon.scope: # v is in one more
                                conflicts
                                var_differential[v] =
                                    var_differential.get(v,0)+1
                self.variable_pq.update_each_priority(var_differential)
                self.display(2,"Number of conflicts",len(self.conflicts))
            if not self.conflicts: # no conflicts, so solution found
                self.display(1,"Solution found:",
                    self.current_assignment,"in",
                            self.number_of_steps,"steps")
                return self.number_of_steps
        self.display(1,"No solution in",self.number_of_steps,"steps",
                    len(self.conflicts),"conflicts remain")
        return None
```

create_pq creates an updatable priority queue of the variables, ordered by the number of conflicts they participate in. The priority queue only includes variables in conflicts and the value of a variable is the *negative* of the number of